Design and Implementation of a Web System for Searching and Managing Courses - Course Finder

Final Report

Brayan Alejandro Riveros Rodríguez
Cristian Santiago Ríos Vásquez
Carlos Andres Pescador Castro

Systems Engineering

Universidad Distrital Francisco José de Caldas

December 11, 2025

Abstract

Students often must search through numerous of courses, multiple instructors, and changing schedules across semesters, but information is often not centralized or in a palatable format, or in partially synchronized systems, making discovery and planning difficult. This project proposes a web-based system that consolidates course data and formalizes instructor-course relationships, separating authentication/authorization services from central domain-based course services, with non-functional objectives to integrate and centralize data, efficient search, and course-instructor associations. Workshops 1 and 2 established the central problem using business model canvas, user stories, CRC cards, and overall system design. Guided by Scrum as a development management methodology, preparing for sprint planning and traceability, this first report summarizes preliminary results and describes the project road map.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Context and Motivation

Students often have no way to organize their semester schedules. At many institutions, including our own, course information is often poorly organized. This fragmentation makes it difficult for students to discover relevant offerings of interest, as well as the professors who teach them and their schedules. The result is avoidable problems during planning and registration periods, duplicate registrations, and delays in decision-making.

To address these issues, we propose a web system for searching and managing academic courses that consolidates essential data (course codes, titles, descriptions) and formalizes the relationships between professors and courses. The system targets three primary stakeholder groups: (i) *students*, who need fast, accurate search and discovery; (ii) *professors*, who is the associated with the courses; and (iii) *administrators users*, which is responsible for loading the data and managing the application. Our current work established the problem framing and initial design assets for the development phase.

From an engineering perspective, the project is intentionally designed to practice delivering end-to-end software within a semiannual schedule by implementing an agile approach.

1.2 Objectives

- Design and specify a reliable web-based system that allows key stakeholders to search, manage and maintain accurate information about courses and instructors.
- Define a clear and feasible road map for the implementation of the project that aligns with the identified problem context, stakeholder needs, and planning.
- Synthesize the context, stakeholders, and scope of the problem into a consistent baseline that unifies terminology and guides subsequent design and development activities.

1.3 Scope

This first report defines the requirements and design boundaries of the project at its current stage. It focuses on consolidating the conceptual and architectural foundations, including problem definition, stakeholder identification, user stories, CRC cards, high-level architecture, deployment views, and early user-interface sketches. The report also introduces preliminary elements for the data model and API contracts that will guide the implementation in the next phase. This scope explicitly includes documenting the

problem context, justifying the proposed solution, and establishing an initial road map under agile methodology using Taiga as the project management tool.

2 Related Work and Background

Students need digital systems to search and manage academic information such as course offerings, instructor assignments, and scheduling. However, on many occasions they find that in fragmented data or centralized in legacy applications that lack interoperability and transparency for stakeholders. In this context, academic management systems and course catalog platforms such as Moodle, Blackboard, and Google Classroom provide partial solutions, focusing primarily on learning management rather than the structural organization of academic offerings.

From a technical point of view, modern web-based architectures rely on Restful API s, micro-service decomposition, and standard authentication mechanisms such as JWT (JSON Web Token) to ensure secure and maintainable information exchange. These principles underpin the design of this project: a lightweight, modular system divided into a authentication service and a business This dual-back-end structure supports scalability, separation of concerns, and parallel development by different team members. Furthermore, adopting a continuous integration and deployment pipeline through tools like Docker and GitHub Actions allows the project to align with current DevOps practices that promote automation, repeatability, and early defect detection.

In terms of methodology, the use of Scrum supported support on the Taiga tool reflects an agile approach, iterative development of software engineering. Agile methods are widely used in both industry and academia to encourage adaptability, transparency, and stakeholder feedback. Previous studies and projects in software engineering have shown that integrating agile practices allows for better quality deliveries.

In summary, this work draws on principles from web-based academic management systems, requirement analysis and software design to propose a simple, fast, and verifiable solution tailored to the needs of the stakeholders.

3 Methodology

3.1 Process and Team Practices

The project will follows an agile software development approach based on the Scrum framework, adapted to the context of problem. The team uses **Taiga** as the main project

management tool to maintain the product backlog, define sprint goals, and track progress through user stories and tasks.

Roles are distributed collaboratively among the three members of the team. Carlos Andres Pescador Castro serve as the Product Owner and is the responsible for prioritizing the backlog and maintaining alignment with the defined requirements. Cristian Santiago Ríos Vásquez act as the Scrum Master, facilitating coordination among team members, ensuring adherence to Scrum events, and managing sprint reviews and retrospectives. Brayan Alejandro Riveros Rodríguez as the Development Lead, focusing on technical implementation, version control integration, ci/cd pipeline and testing coordination. All members contribute equally to documentation, gathering of requirements and continuous improvement activities.

Scrum ceremonies are conducted at a scaled frequency appropriate brief stand-up meetings are held twice per week (asynchronous when needed), sprint planning and retrospectives occur at the beginning and end of each workshop, and sprint reviews coincide with workshop submissions.

Version control and collaboration are managed through a shared Git repository, following a feature-branch workflow.

3.2 Scrum Implementation and Project Management

a

3.3 System Analysis and Design

Following the completion of the Workshops, the system analysis and design phase guide the transition from requirements specification to technical implementation. The main goal will be to ensure that the proposed architecture, domain entities, and component responsibilities are consistent with the user stories, acceptance criteria, and stakeholder needs identified during the analysis stage.

The analysis focus on consolidating the functional requirements derived from the user stories created in Workshop 1. Each user story will be mapped to a corresponding set of system components, defining clear inputs, processes, and outputs. CRC (Class–Responsibility–Collaborator) cards previously designed will serve as the foundation for assigning responsibilities among classes, while maintaining object-oriented design principles. This mapping will guarantee that each functional requirement has a direct

representation within the software model.

The system will adopt a modular architecture divided into two main backend services: an authentication and authorization service, and a core domain service for managing courses, professors, and their relationships. Both services will communicate through RESTful APIs, enabling independent deployment and scalability. Each backend will interact with its respective database—MySQL for authentication and PostgreSQL or MongoDB for the core domain—ensuring persistence and referential integrity across entities. The frontend web interface, developed with modern JavaScript framework, will consume these APIs to provide a cohesive user experience.

To formalize the design, UML diagrams are used to illustrate the system structure and interactions. A class diagram will describe the main domain entities (Professor, Course, and Link), their attributes, and associations. A deployment diagram will detail how components will be distributed across environments, showing the web client, API services, and database instances.

Non-functional requirements will also guide the design. The system will ensure security through JWT-based authentication and role-based access control, maintainability through modular coding practices, and performance through optimized database queries and API endpoints. Furthermore, logging and basic monitoring mechanisms will be defined to support future testing and evaluation.

This analysis and design process will establish a clear technical blueprint for the implementation stage to be developed in Workshop 3. By grounding each architectural decision in the previously validated requirements, the team will ensure traceability from stakeholder needs to system components and a solid foundation for the subsequent development and testing phases.

4 System Design and Architecture

4.1 User Stories and Requirements

The functional requirements of the system were captured through user stories following agile methodology principles. Each story represents a specific user need and includes well-defined acceptance criteria that guide both development and testing activities. The stories were structured using the standard template: "As a [user role], I want to [perform action], so that [achieve benefit]." This approach ensures that every feature delivers clear value to stakeholders and remains traceable throughout the development lifecycle.

Five core user stories form the foundation of the system functionality, divided into administrative capabilities, end-user search features, and role-based navigation. These stories were prioritized based on their dependency relationships and their impact on delivering value to all stakeholder groups. Table 1 provides a summary of all user stories with their dependencies and roles.

Table 1: Core User Stories for System Functionality

Story ID	**Story Name**	**User Role**	**Dependencies**
US_CF_01	Registration of New Professors	System Administrator	None
US_CF_02	Registration of New Courses	System Administrator	None
US_CF_03	Link Professor to Courses	System Administrator	US_CF_01, US_CF_02
US_CF_04	Search Courses by Professor and Vice Versa	System User	US_CF_03
US_CF_05	Home Page and Access Flow	Administrator /General User	US_CF_04, Auth Module

4.1.1 Administrative User Stories

Three stories address the System Administrator role, enabling the creation and management of the academic catalog that supports the search functionality.

US_CourseFinder_01: Registration of New Professors This story addresses the need to register professors in the system by capturing their personal and academic details through a dedicated registration form.

User Need: The administrator requires the ability to add new professors to the platform by entering their personal and academic information, ensuring the professor database remains current and complete.

Justification: Maintaining an up-to-date professor database ensures that users have access to accurate and complete information when consulting the system, supporting informed decision-making during course selection.

Key Validations: The system must validate all required fields before persisting data, including name, identification number, email, and specialty. Email uniqueness must be enforced to prevent duplicate registrations. The identification number must also be unique across all professor records.

Acceptance Criteria: The administrator can access the registration form from the main admin panel. All required fields are validated before saving data. The system displays a confirmation message upon successful registration. The new professor appears in the general professor list without requiring a page reload. Duplicate registrations based on email or identification number are prevented with appropriate error messages.

INVEST Characteristics: This story is independent and does not depend on other stories for development. It is negotiable in terms of specific fields and validation rules, which can be adjusted according to academic requirements. The story provides immediate value by establishing the foundational data required for the platform. Development effort is estimable based on form complexity and validation logic. The scope is small enough to complete within a single sprint. The story is testable through field validation tests, database persistence verification, and UI confirmation message checks.

US_CourseFinder_02: Registration of New Courses This story enables administrators to expand the academic catalog by adding new courses with comprehensive metadata.

User Need: The administrator wants to add new courses to the system, including details such as course name, code, description, credits, and category.

Justification: Expanding and maintaining the academic offerings ensures users have access to current course information, supporting effective academic planning and course discovery.

Key Validations: The system must validate course code uniqueness to prevent duplicate entries. All required fields must be completed before submission. If a professor is assigned during course creation, the relationship must be validated to ensure the professor exists in the database.

Acceptance Criteria: The administrator can access the course registration form from the main panel. All required fields are validated before saving. A confirmation message appears upon successful registration. The new course appears in the course list without

page reload. The system prevents creation of duplicate courses based on course code. If a professor is assigned, the relationship is correctly recorded in the database.

INVEST Characteristics: This story is independent and can be developed without dependencies on other stories. Fields and professor relationships are negotiable based on academic system needs. The story expands the information base, increasing platform utility for end users. Effort is estimable considering form implementation, validations, and database operations. The scope fits within a single sprint. Testing validates field requirements, database persistence, and relationship integrity.

US_CourseFinder_03: Link Professor to One or Multiple Courses This story implements the core relationship management functionality that establishes the many-to-many associations between professors and courses.

User Need: The administrator wants to link existing professors to one or more existing courses, establishing the teaching assignments that users will query.

Justification: Correct professor-course associations enable users to view accurate relationships when searching, supporting informed course selection based on instructor preferences.

Key Validations: Both the professor and course must exist in the database before linking. The system should prevent duplicate assignments of the same professor-course-group combination. If a course is already assigned to another professor in the same group, the system should warn the administrator or prevent the duplication based on business rules.

Acceptance Criteria: The administrator can access a dedicated interface for linking professors to courses. The system displays only existing professors and courses. The administrator can select one professor and one or multiple courses to link. Upon confirmation, the links are stored and visible in both entity profiles. Error messages appear if attempting to link non-existent entities. The system prevents or warns about duplicate course assignments. The administrator can unlink or modify existing relationships. A confirmation message appears after successful linking.

INVEST Characteristics: This story depends on US_CF_01 and US_CF_02, as both entities must exist before linking. The user interface and workflow are negotiable based on administrator feedback. The story provides core administrative control over relationships, ensuring data accuracy. Development time is estimable based on form

complexity, relationship logic, and validations. The scope fits within a single sprint for a small team. Testing validates UI interactions and backend relationship integrity.

4.1.2 End-User Functionality

Two stories address the primary use cases for general users: discovering course and professor information through search and accessing the system through role-appropriate entry points.

US_CourseFinder_04: Search Courses by Professor and Professors by Course
This story provides the core bidirectional search functionality that motivated the project development.

User Need: System users want to search for courses taught by a specific professor and search for courses to view their assigned professors.

Justification: Easy access to accurate professor-course relationships improves navigation and course discovery, helping users make informed academic decisions efficiently.

Key Validations: Search queries must handle partial text matches and ignore case sensitivity. The system must validate that search results accurately reflect the professor-course relationships established by administrators. Results must be complete and consistent with the database state.

Acceptance Criteria: Users can search by professor name and view all associated courses. Users can search by course name and view all assigned professors. The system displays clear, accurate, and complete results for each query. Search supports partial text matching and is case-insensitive. Friendly messages appear when no results are found. Search performance remains under two seconds for average query loads. The interface is responsive and adapts to desktop and mobile devices.

INVEST Characteristics: This story depends on US_CF_03, as accurate search requires established professor-course associations. Search filters, display format, and pagination are negotiable based on usability feedback. The story provides core functionality for navigating relationships, improving user experience significantly. Effort is estimable based on filter implementation, database queries, and frontend integration. The scope fits within a single sprint for a small development team. Testing validates that results accurately match linked data between professors and courses.

US_CourseFinder_05: Home Page and Access Flow for Administrator and User This story defines the main entry point of the system, providing role-appropriate navigation paths for administrators and general users.

User Need: Administrators want to access a dashboard summarizing key metrics with shortcuts to management actions. General users want immediate access to the search feature without unnecessary navigation.

Justification: Role-based entry points enable both user types to efficiently access the system according to their needs—administrators can manage data quickly, and users can immediately start searching.

Key Validations: The system must correctly identify the user's role after login. Role-based routing must function reliably, directing administrators to the dashboard and general users to the search interface. The homepage must load within acceptable performance thresholds (under two seconds). Administrative elements must be completely hidden from general user views.

Acceptance Criteria: The system redirects users based on their role: administrators to the dashboard, general users to the search page. The homepage loads correctly and displays personalized content. The admin dashboard shows relevant metrics (total professors, total courses, recent updates) and quick management options. The general user view focuses exclusively on search functionality. The interface is responsive and loads within performance thresholds. Navigation between homepage and other sections works without errors.

INVEST Characteristics: This story depends on US_CF_04 and the authentication module, as it requires functional search capabilities and role identification. Homepage layout, design elements, and dashboard widgets are negotiable based on feedback and usability tests. The story provides personalized entry points, ensuring efficient access and navigation for both roles. The scope is measurable by estimating layout design, routing logic, and conditional rendering. Implementation fits within a single sprint. Testing verifies redirection logic, role-based rendering, and proper section access.

4.1.3 Story Management and Traceability

The user stories were managed in the Taiga project management platform, where they were assigned story points, prioritized in the product backlog, and allocated to specific

sprints during sprint planning sessions. Each story's progress was tracked through states (New, In Progress, Done) and linked to specific tasks. Story points were estimated using planning poker sessions, with the team achieving consensus on complexity based on technical effort, uncertainty and integration requirements.

The dependency chain from administrative stories through search functionality to role-based navigation guided sprint planning, ensuring that foundational data management capabilities were delivered before user-facing features. This traceability ensures that every implemented feature can be validated against its original acceptance criteria and that test coverage aligns with user expectations. The acceptance criteria defined in each story directly influenced the test scenarios documented in Section 5 and the evaluation metrics presented in Section 6.

4.2 Web User Interface

This section presents the main artifacts developed during all the Workshops, which represent the foundation for the system implementation. Each artifact serves a specific purpose in documenting the design decisions and validating the feasibility of the proposed solution. The progression from conceptual models to concrete specifications ensures traceability from requirements to implementation.

Figure 1: Web User Interface (v1.0)

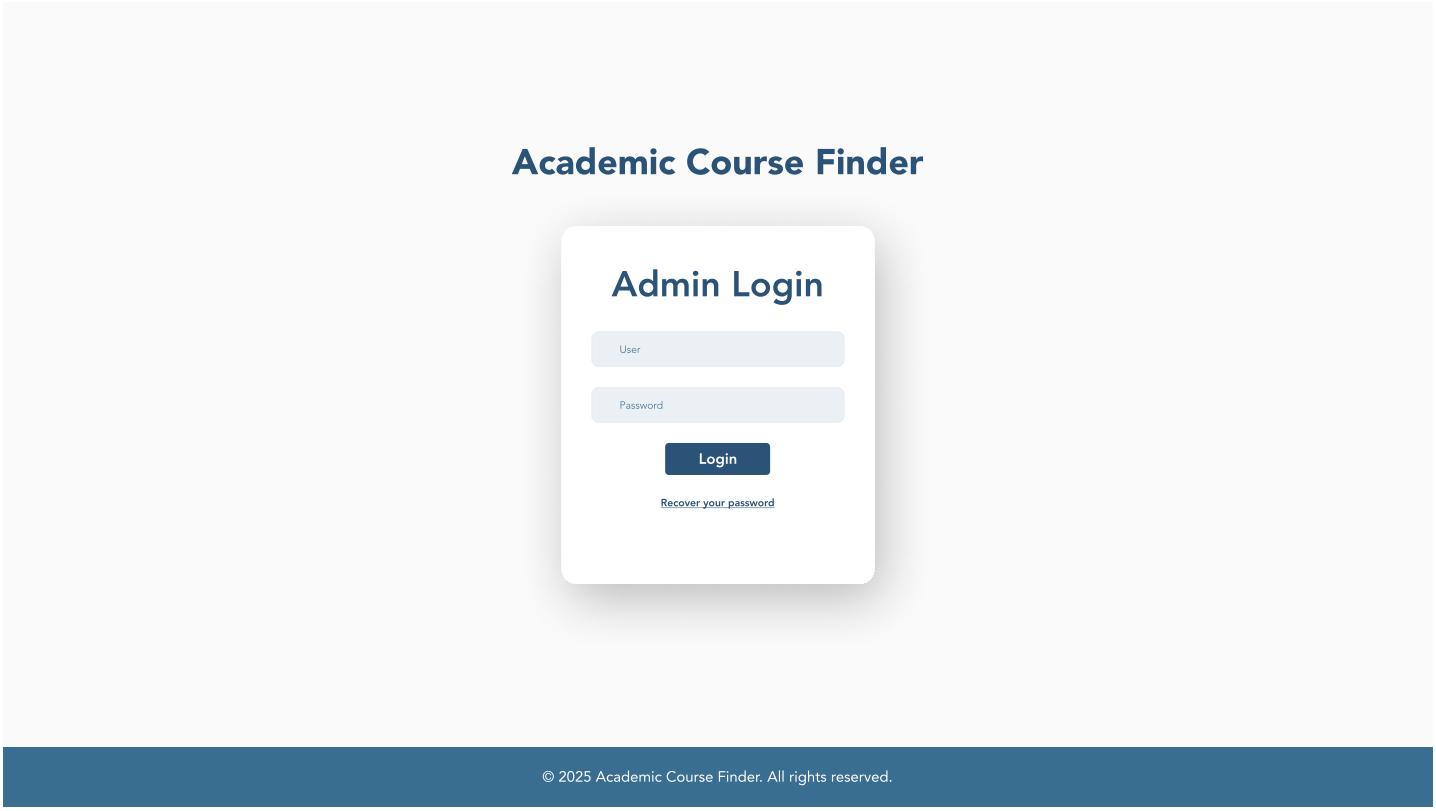


Figure 2: Web User Interface (v1.0)

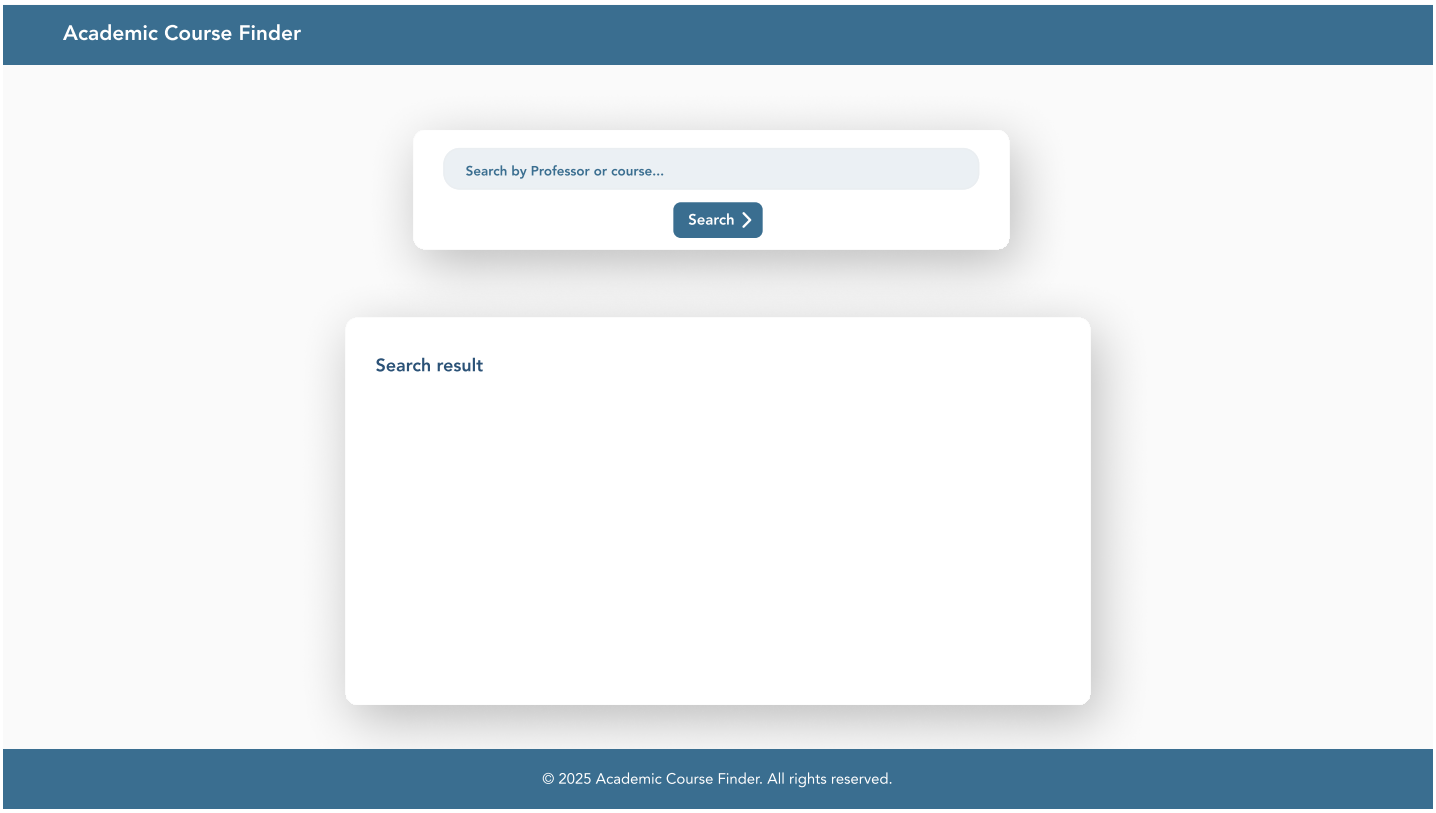


Figure 3: Web User Interface (v1.0)

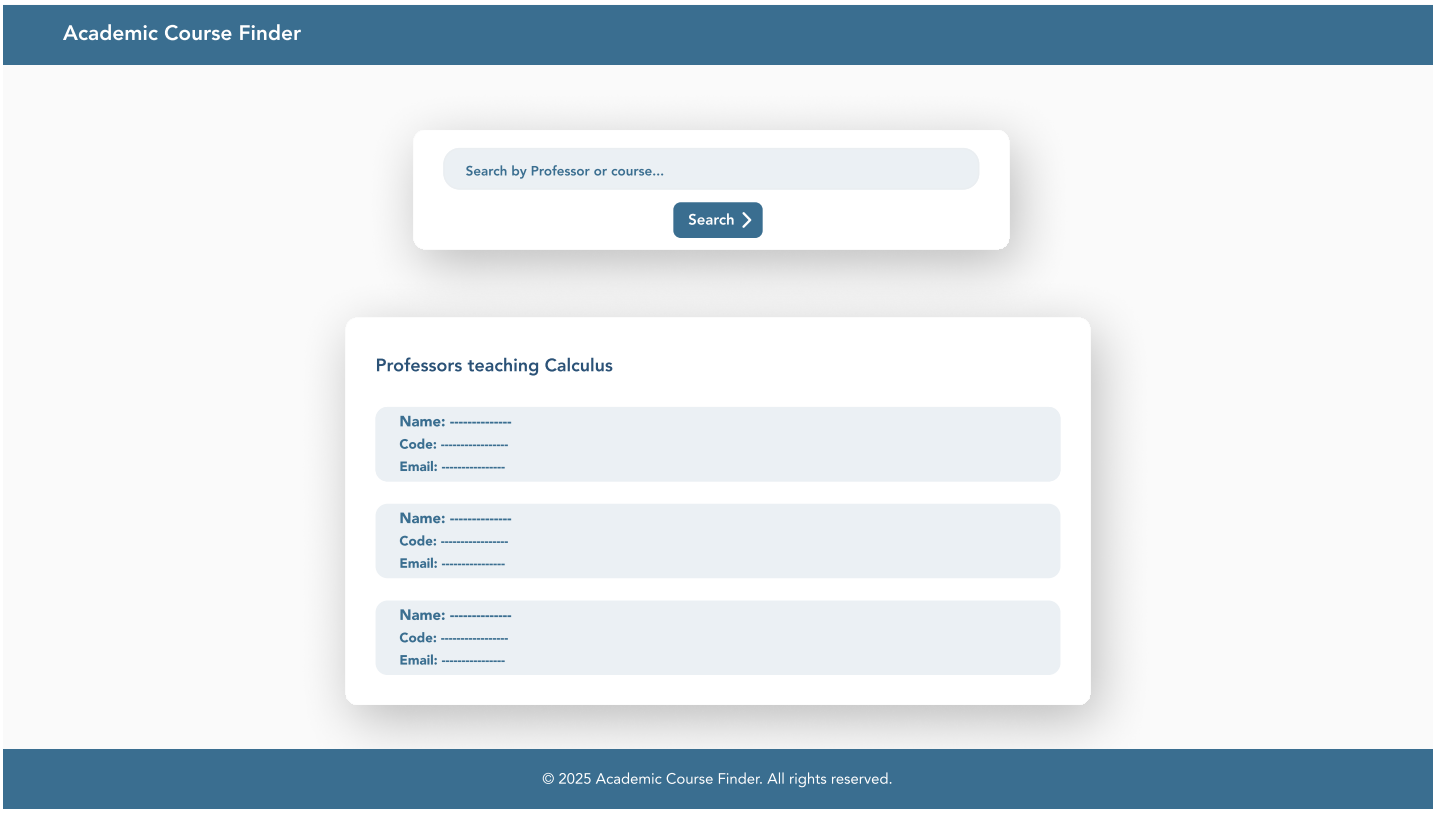


Figure 4: Web User Interface (v1.0)

Figure 5: Web User Interface (v1.0)

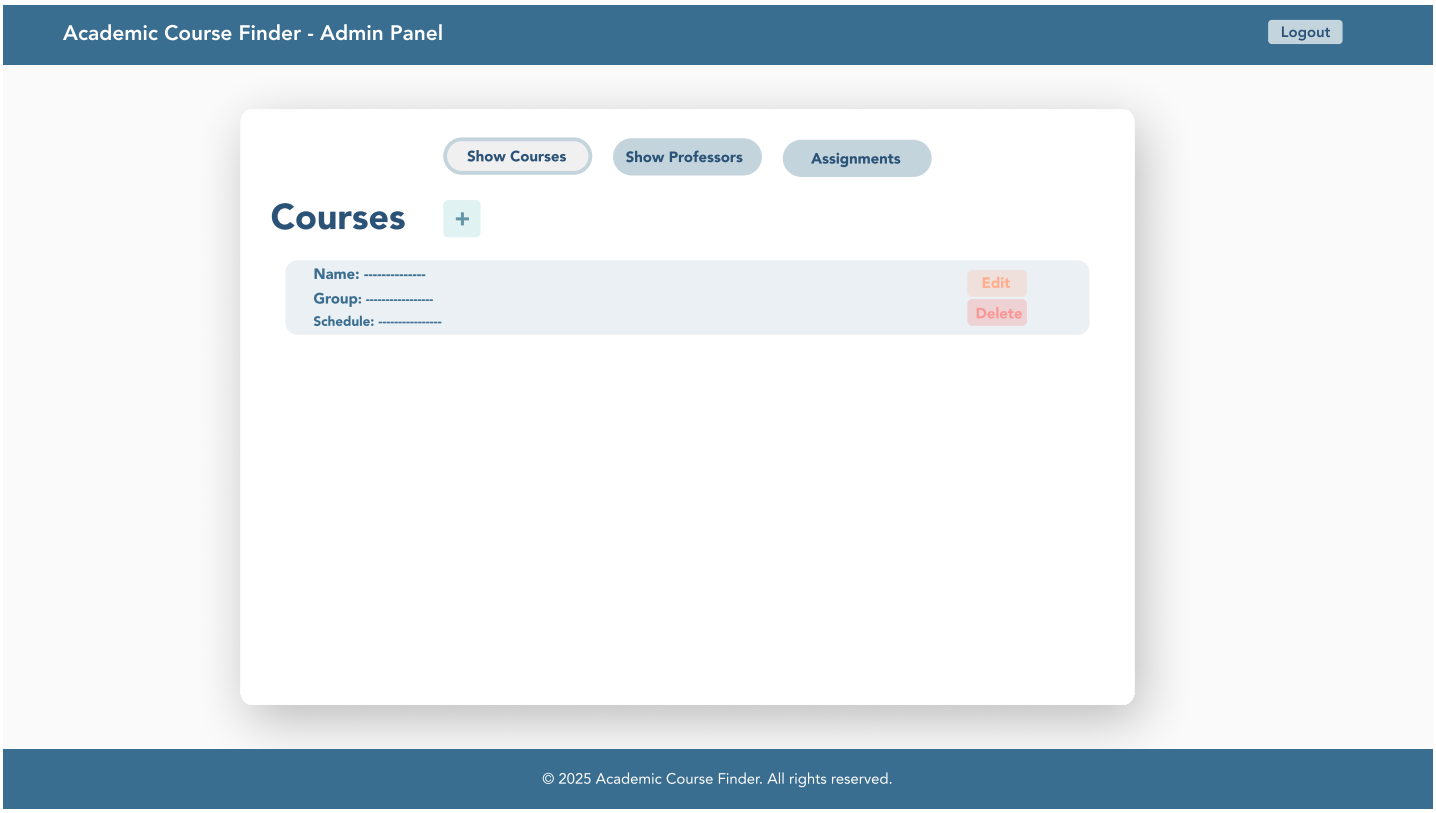
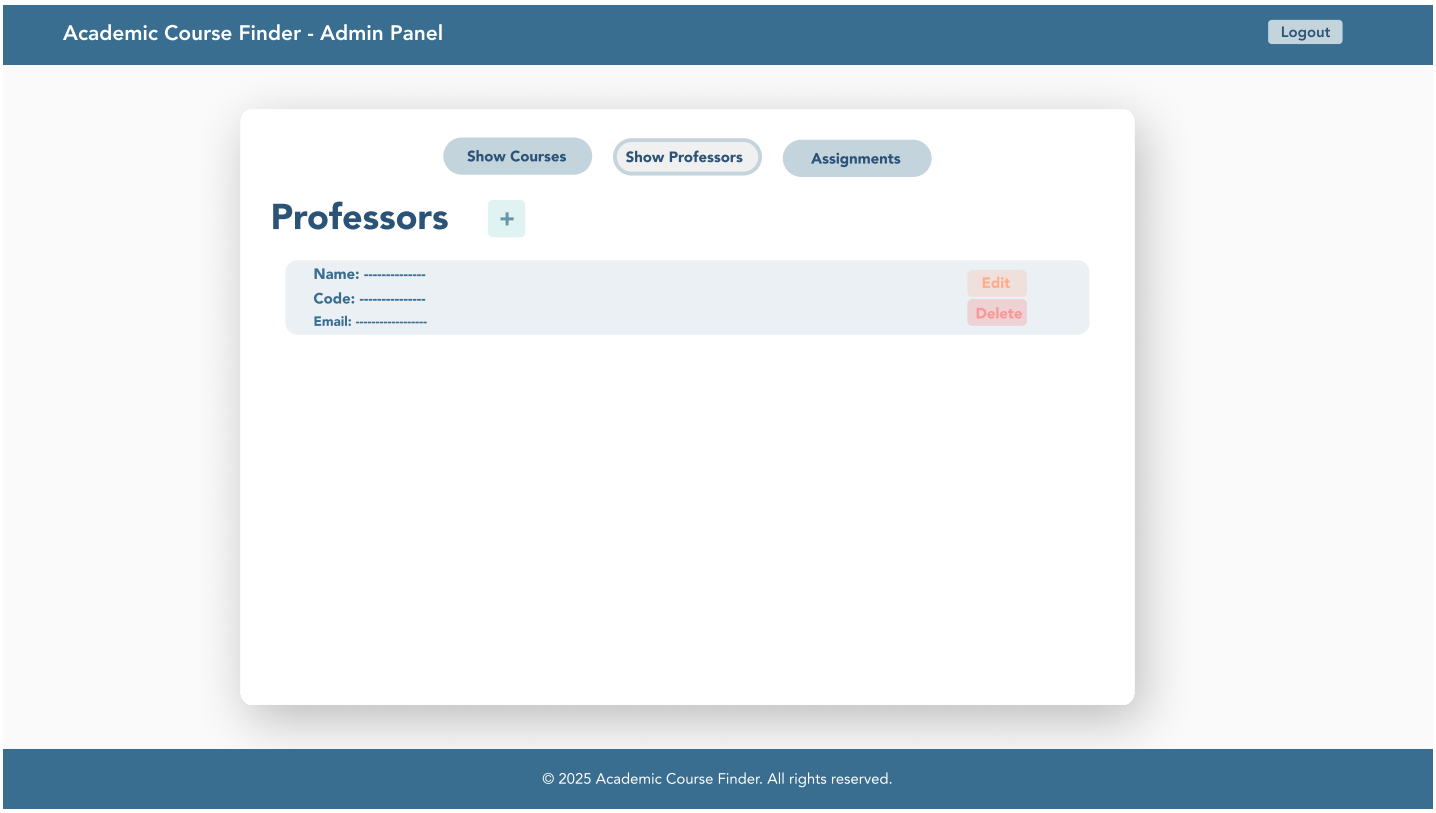


Figure 6: Web User Interface (v1.0)



The user interface was designed following the principle "Simple. Clear. Consistent." The visual design prioritizes clarity, usability, and role-based access control to ensure that each stakeholder group can efficiently accomplish their tasks without unnecessary complexity. Six main views were designed and subsequently implemented in the production environment, representing the complete user journey through the system.

The interface accommodates three distinct user roles with different access levels and capabilities. Anonymous users can access the search functionality, allowing them to query courses by code, title, or professor details without authentication. The search results display comprehensive course information including schedules and associated instructors. Authenticated administrators access additional views for data management, including CRUD operations for courses and professors, as well as the assignment interface that links professors to specific courses and groups.

Design decisions emphasized consistency across views through standardized navigation patterns, uniform typography, and coherent color schemes that distinguish between public and administrative sections. The responsive layout adapts to different screen sizes, ensuring accessibility across desktop and mobile devices. Form validation provides immediate feedback to users, reducing errors during data entry. The role-based design ensures that administrative functions remain hidden from unauthorized users while maintaining an intuitive experience for all stakeholder groups.

The previous figures presents the final implemented views of the web application, demonstrating the translation from initial mockups to functional interfaces. These views validate the usability requirements established during the requirements analysis phase and confirm that the system meets the interaction needs identified in the user stories.

4.3 Architecture Overview

The system architecture implements a microservices approach with clear separation of concerns across three main components: frontend, an authentication service and a business logic service. This architectural decision was mandated by project requirements and offers several strategic advantages for development, deployment, and maintenance.

The separation of authentication into an independent service provides security isolation, ensuring that credential management and token generation occur in a dedicated, hardened environment. This service connects to a database optimized for transaction processing and user session management. The authentication service issues JWT tokens upon successful login, which are subsequently validated by the business logic service for

protected endpoints. This token-based approach enables stateless authentication, simplifying horizontal scaling and load balancing.

The business logic service handles all domain operations including course management, professor administration, search functionality, and relationship assignments. By isolating these operations in a separate service connected to a database different from the one used for authentication, the system achieves better performance through database specialization.

Component communication follows RESTful principles through well-defined API contracts. The frontend makes HTTPS requests to both backend services, with the authentication service providing tokens and the business service consuming them for authorization. This loose coupling allows independent deployment and version management of each service. Furthermore, the architecture supports parallel development, enabling team members to work simultaneously on different services without integration conflicts.

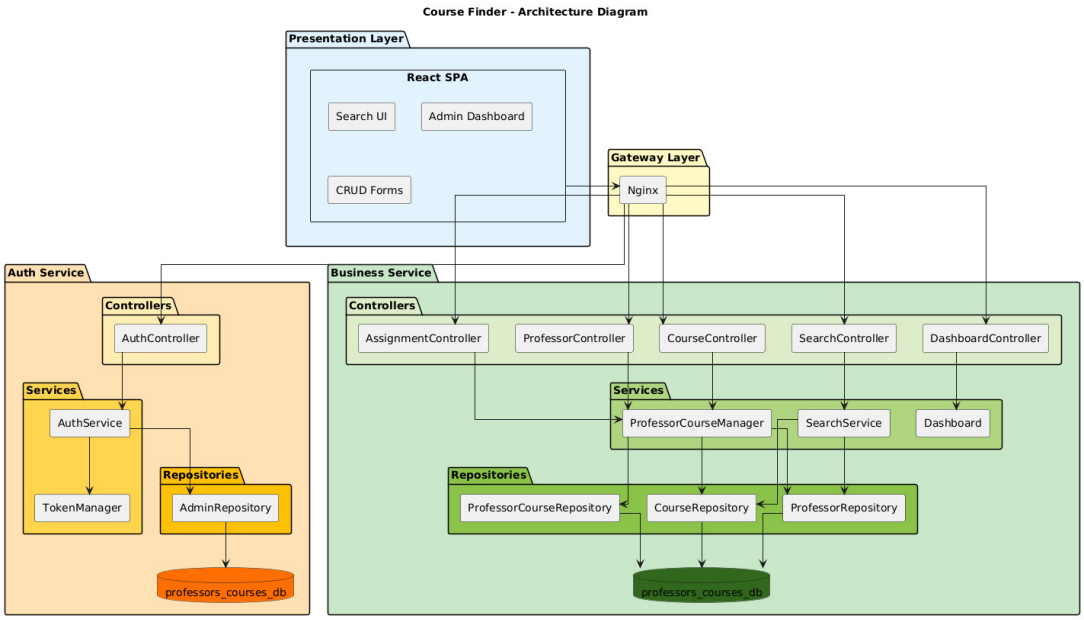


Figure 7: Architecture Diagram (v1.0).

Figure 7 illustrates the high-level architecture, showing the relationships between components, the data flow from user interaction through business logic to persistence layers, and the external interfaces exposed by each service. This diagram serves as the reference model for understanding system boundaries and component responsibilities throughout the development process.

4.4 Data Model and Database Schema

The data model defines four primary entities that capture the essential academic domain: Professor, Course, Admin, and ProfessorCourse. This structure was designed to support the core user requirements identified during the analysis phase, particularly the need to search courses by multiple criteria and maintain clear associations between professors and their teaching assignments.

4.4.1 Domain Entities and Relationships

The Professor entity contains identification attributes (id, name) and academic metadata (email, specialty) that categorizes the instructor's area of expertise. This information enables filtering and grouping operations in search queries. The email field is enforced as unique to prevent duplicate professor registrations and serves as a natural key for external integrations.

The Course entity includes structural attributes (id, code, name) along with scheduling information (schedule) that supports both course discovery and academic planning. The unique course code constraint ensures data integrity and prevents duplicate registrations. The schedule field stores time and day information in a flexible text format, allowing representation of various scheduling patterns without imposing rigid structural constraints.

The relationship between professors and courses implements a many-to-many association through the ProfessorCourse junction entity. This design decision reflects the reality of academic scheduling where one professor may teach multiple courses and one course may be taught by multiple professors across different groups or semesters. The ProfessorCourse entity includes additional context such as assigned_at (timestamp of assignment creation) and status (to manage active, inactive, pending, or completed assignments). This rich association model provides the flexibility required for complex academic scenarios while maintaining referential integrity through foreign key constraints with cascade delete behavior.

The Admin entity remains architecturally separate from the academic domain, containing only authentication-relevant attributes (id, username, password_hash, email). This separation ensures that administrative access control does not interfere with the core domain logic and allows for independent security policy management. The Admin entity resides in a separate database dedicated to the authentication service, while the academic entities (Professor, Course, ProfessorCourse) reside in a database managed by the business logic service.

Design decisions prioritized query performance for the primary use case: searching courses by professor or course attributes and retrieving all related information in minimal database operations. The normalized structure prevents data redundancy while the junction entity enables efficient joins. Repository interfaces abstract database operations, providing a clean separation between domain logic and persistence mechanisms that facilitates future database migration or optimization.

4.4.2 Business Logic Database

The business logic database implements the academic domain model. Table 2 summarizes the three main tables with their key attributes and constraints.

Table 2: Schema - Academic Domain Tables

Table	Key Attributes	Constraints
`professors`	id (SERIAL), name, email, specialty, created_at, updated_at	UNIQUE(email), Indexes on email, name, specialty
`courses`	id (SERIAL), name, code, schedule, created_at, updated_at	UNIQUE(code), Indexes on code, name, schedule
`professor_courses`	id (SERIAL), professor_id, course_id, assigned_at, status	FK to professors/courses, UNIQUE(professor_id, course_id), CHECK status IN ('active', 'inactive', 'pending', 'completed')

The `professors` table stores instructor information with automatic timestamp tracking. Indexes on email, name, and specialty optimize common query patterns including professor search by name and filtering by academic area. The email uniqueness constraint prevents duplicate registrations and supports using email as a natural identifier in external systems.

The `courses` table maintains course catalog information with indexed fields supporting fast lookup by course code or name. The schedule field stores time and location information in a flexible VARCHAR format, allowing various representations ("Mon/Wed 10:00-12:00", "Tue/Thu 14:00-15:30") without rigid parsing requirements. The unique code constraint ensures each course has a distinct identifier within the institution.

The `professor_courses` junction table implements the many-to-many relationship with additional assignment metadata. Foreign key constraints with CASCADE DELETE ensure referential integrity—deleting a professor or course automatically removes associated assignments, preventing orphaned records. The composite unique constraint on (professor_id, course_id) prevents duplicate assignments of the same professor to the same course. The status field uses a CHECK constraint limiting values to predefined states, supporting assignment lifecycle management. Multiple indexes optimize queries retrieving courses by professor, professors by course, and filtering by assignment status. A composite index on (professor_id, course_id, status) accelerates the most common query pattern: finding active courses for a specific professor.

Database triggers automatically update the `updated_at` timestamp on all tables whenever records are modified, providing audit trail capabilities without application-level logic. This pattern ensures consistent timestamp management across all domain entities.

4.4.3 Authentication Database

The authentication service uses a separate database (`auth_db_course`) to isolate security-critical data from business logic. Table 3 describes the authentication schema structure.

Table 3: Schema - Authentication Table

Table	Key Attributes	Constraints
`admins`	id (INT AUTO_INCREMENT), username, password_hash, email, created_at, updated_at	UNIQUE(username), UNIQUE(email), Indexes on username and email

The `admins` table stores administrative user credentials with bcrypt-hashed passwords in the `password_hash` field. Both username and email are enforced as unique, providing multiple authentication options. Indexes on both fields optimize login queries, which typically search by username.

The deliberate separation of authentication and business databases supports the microservices architecture by allowing independent scaling, backup strategies, and security policies. The authentication database requires higher security measures while the business database optimizes for read-heavy query patterns typical of search operations.

4.4.4 Schema Design Rationale

Several design decisions warrant explicit justification. First, the choice of different database systems (PostgreSQL for business logic, MySQL for authentication) was mandated by project requirements but offers strategic advantages. PostgreSQL's advanced JSON support, full-text search capabilities, and complex query optimization make it well-suited for the search-intensive academic domain. MySQL's simpler deployment and mature authentication integrations align with the lightweight requirements of the authentication service.

Second, the denormalized storage of schedule information as text rather than structured time slots prioritizes implementation simplicity and flexibility over query complexity. While this approach limits schedule-based filtering capabilities, it satisfies current requirements and can be refactored if advanced schedule analysis becomes necessary.

Third, the status field in `professor_courses` supports future features like temporary assignments, historical tracking, and workflow management without schema changes. The CHECK constraint ensures data quality while allowing extension through configuration rather than migration.

Finally, comprehensive indexing strategies reflect analysis of expected query patterns: searches by professor name, searches by course code/name, and retrieval of professor-course relationships. Composite indexes optimize multi-condition queries while single-column indexes support individual lookups. This indexing approach balances query performance against write overhead and storage costs.

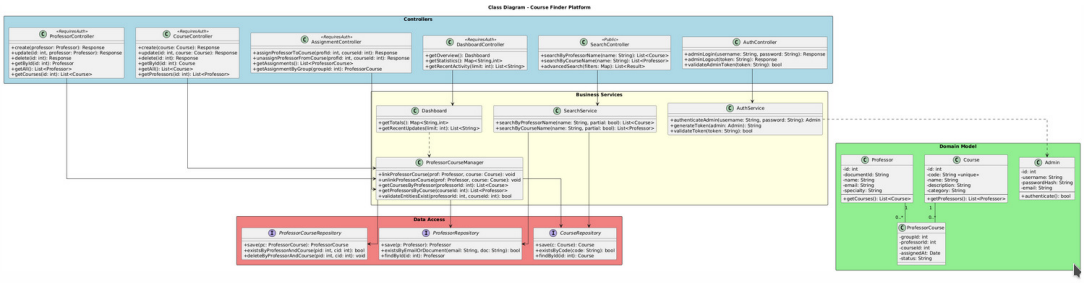


Figure 8: Class diagram showing domain entities, repositories, services, and controllers across all architectural layers.

Figure 8 presents the complete class diagram including entities, repositories, services, and controllers across all architectural layers. This diagram demonstrates how the data model integrates with business logic components and exposes functionality through controller endpoints. The layered structure visible in the diagram reinforces the architectural principles of separation of concerns and dependency management established in the sys-

tem design. The repository interfaces shown in the diagram directly correspond to the database tables documented in this section, providing the abstraction layer that enables database-agnostic business logic implementation.

4.5 API Endpoints and Contracts

The system exposes its functionality through two RESTful API services that communicate using JSON payloads over HTTP/HTTPS protocols. Each service provides a well-defined set of endpoints documented through OpenAPI specifications and accessible via Swagger UI for interactive exploration and testing. This section describes the API contracts, request/response formats, and authentication mechanisms that enable communication between the frontend and backend services.

4.5.1 Authentication Service API

The authentication service, built with Java Spring Boot and running on port 8080, provides JWT-based authentication for administrative users. The API documentation is accessible through Swagger UI at `http://localhost:8080/swagger-ui/index.html` during development, offering an interactive interface to explore endpoints, test requests, and view response formats.

Login Endpoint The primary authentication endpoint validates administrator credentials and issues JWT tokens for subsequent authorized requests.

Endpoint: `POST /api/auth/login`
Description: Authenticates an administrator by validating username and password against stored credentials in the MySQL database. Upon successful authentication, the service generates a JWT token containing the admin's identity and role claims.

Figure 9 shows the Swagger UI documentation for the authentication service, illustrating the login endpoint specification with its request schema and expected responses. The interactive interface allows developers to test authentication directly from the browser by providing sample credentials and observing the token generation process.

Figure 10 demonstrates the request format for the login endpoint. The request body requires two fields: username and password, both as string values. The Swagger interface validates the request schema before submission, ensuring that required fields are present and properly formatted.

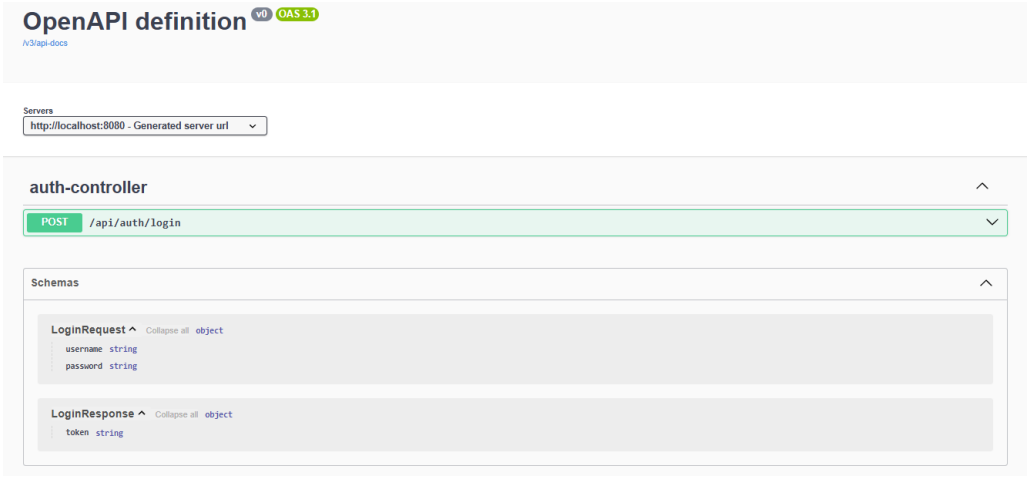


Figure 9: Swagger UI documentation for the authentication service showing the login endpoint specification with request/response schemas.

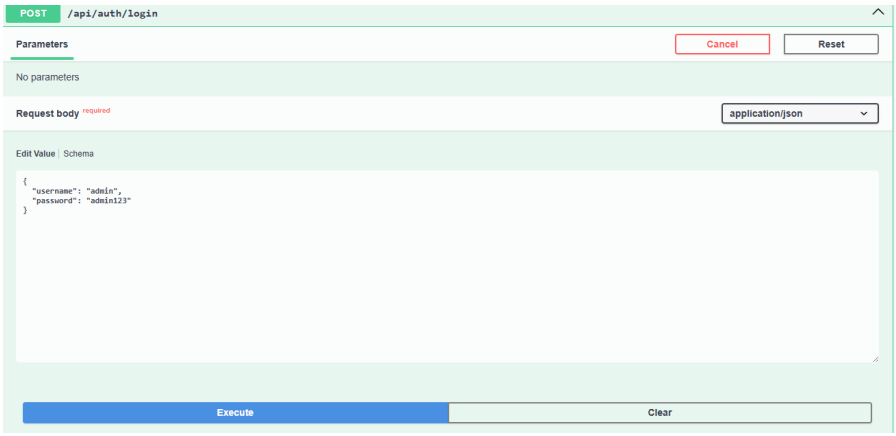


Figure 10: Login endpoint request example showing the JSON payload structure with username and password fields.

Figure 11 shows the successful response format. Upon valid credentials, the service returns a JSON object containing a JWT token string. This token must be included in the Authorization header of subsequent requests to protected endpoints in both services. The token format follows the Bearer authentication scheme: `Authorization: Bearer <token>`. Tokens expire after a configured period (typically 24 hours) and must be refreshed through re-authentication.

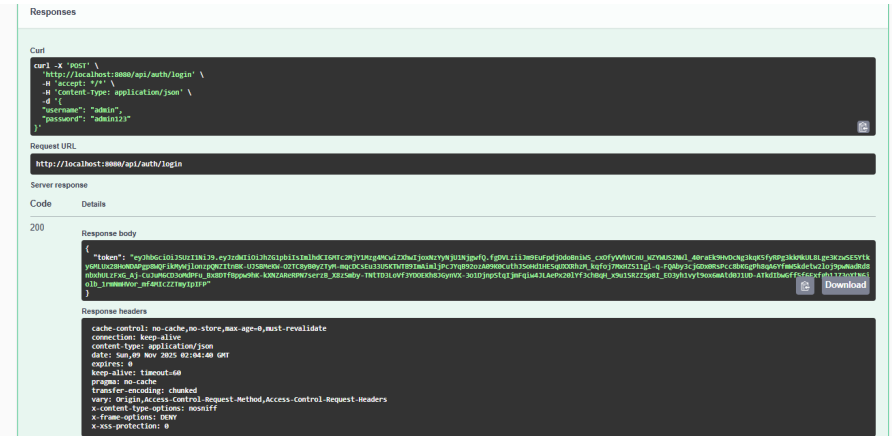


Figure 11: Successful login response showing the generated JWT token that must be used for authenticated requests.

Error responses return appropriate HTTP status codes with descriptive messages. A 401 Unauthorized status indicates invalid credentials, while 400 Bad Request indicates malformed request payloads missing required fields.

4.5.2 Business Logic Service API

The business logic service, built with Python FastAPI and running on port 8000, provides comprehensive CRUD operations for professors, courses, and their assignments. All administrative endpoints require valid JWT tokens obtained from the authentication service. The API documentation is accessible at `http://localhost:8000/docs` during development.

The API follows RESTful conventions with consistent HTTP method semantics: POST for creation, GET for retrieval, PUT for updates, and DELETE for removal. All endpoints return JSON responses with appropriate HTTP status codes (200 for success, 201 for creation, 400 for validation errors, 401 for authentication failures, 404 for not found, 500 for server errors).

Figure 12 presents the comprehensive Swagger UI documentation for the business logic

service, showing all available endpoints organized by resource type (professors, courses, assignments). The interface provides expandable sections for each endpoint with detailed parameter descriptions, request schemas, and response models.

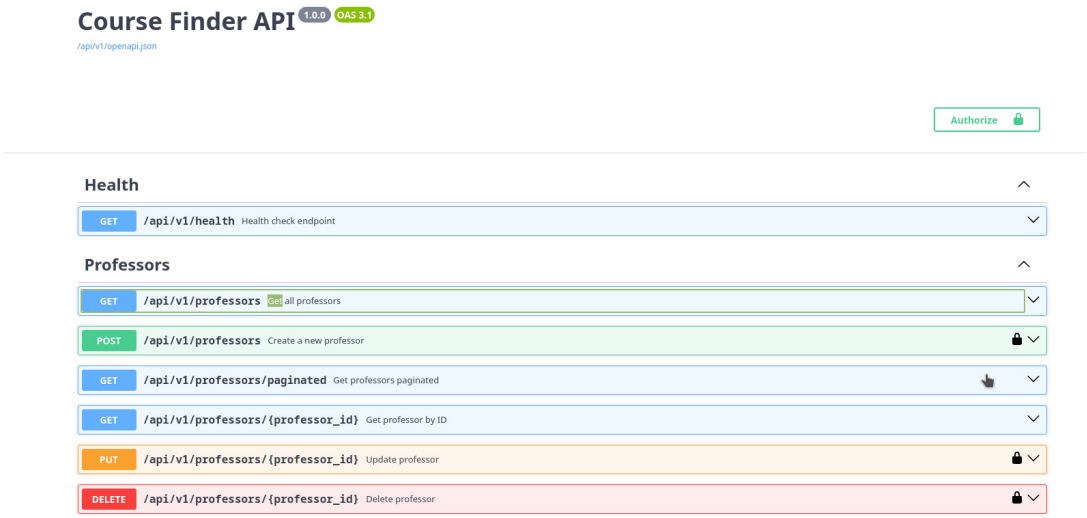


Figure 12: Swagger UI overview of the business logic service showing all CRUD endpoints organized by resource (professors, courses, assignments).

Professor Endpoints The professor endpoints manage instructor data and support the administrative user stories related to professor registration and management.

- `POST /api/v1/professors` — Create a new professor. Requires authentication. Validates email uniqueness and required fields (name, email, specialty). Returns 201 Created with the created professor object including the assigned ID.
- `GET /api/v1/professors` — List all professors with pagination support. Accepts query parameters: `page` (default: 1), `limit` (default: 10), `search` (optional text filter). Returns paginated results with total count and page metadata.
- `GET /api/v1/professors/{id}` — Retrieve a specific professor by ID. Returns 404 if professor not found. Response includes all professor attributes and a list of assigned courses.
- `PUT /api/v1/professors/{id}` — Update professor information. Requires authentication. Validates email uniqueness excluding the current professor. Returns updated professor object.
- `DELETE /api/v1/professors/{id}` — Delete professor. Requires authentication. Cascade deletes all professor-course assignments due to foreign key constraints. Returns 204 No Content on success.

Figure 13 shows the Swagger documentation for the professor creation endpoint. The interface displays the required request body schema with field descriptions, data types, and validation constraints. The example section provides a sample payload that developers can modify and submit directly through the Swagger UI for testing purposes.

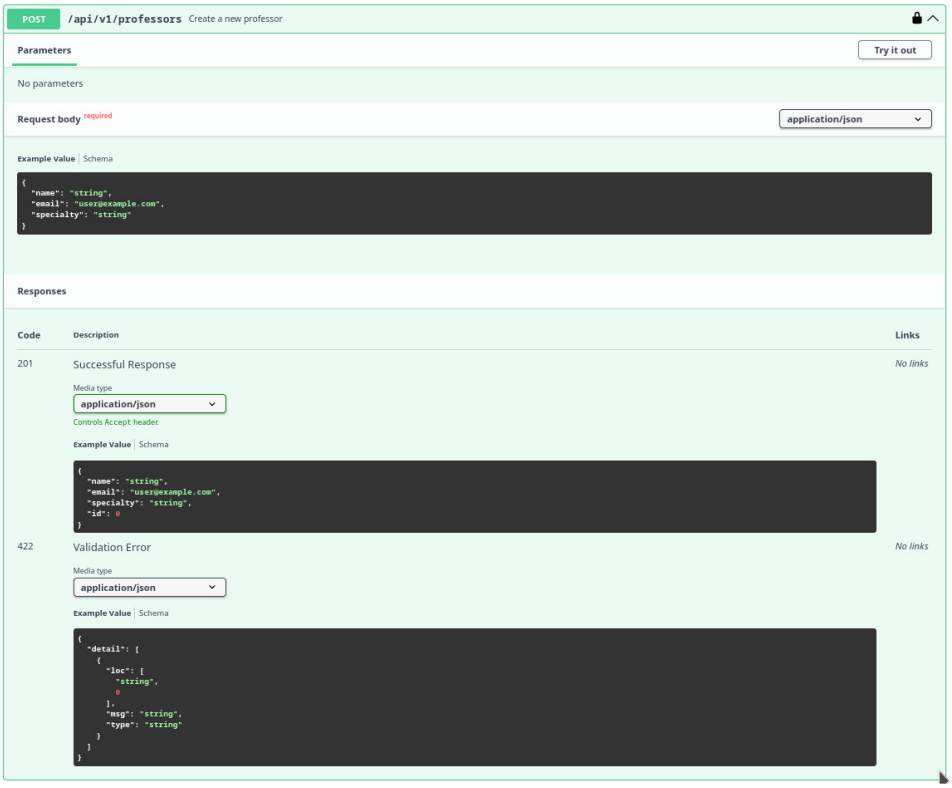


Figure 13: Professor creation endpoint documentation showing request schema with required fields (name, email, specialty) and expected response format.

Course Endpoints The course endpoints manage academic course catalog data, supporting course registration and discovery functionality.

- `POST /api/v1/courses` — Create a new course. Requires authentication. Validates code uniqueness and required fields (name, code). Schedule is optional. Returns 201 Created with the created course object.
- `GET /api/v1/courses` — List all courses with pagination and search capabilities. Accepts query parameters: `page`, `limit`, `search` (filters by course name or code). Returns paginated results.
- `GET /api/v1/courses/{id}` — Retrieve specific course by ID. Returns course details including all assigned professors with their assignment status.

- `PUT /api/v1/courses/{id}` — Update course information. Requires authentication. Validates code uniqueness excluding the current course. Returns updated course object.
- `DELETE /api/v1/courses/{id}` — Delete course. Requires authentication. Cascade deletes all professor-course assignments. Returns 204 No Content on success.

Figure 14 illustrates the complete set of course endpoints available in the Swagger documentation. Each endpoint section can be expanded to reveal detailed parameter information, authentication requirements, and example request/response payloads.

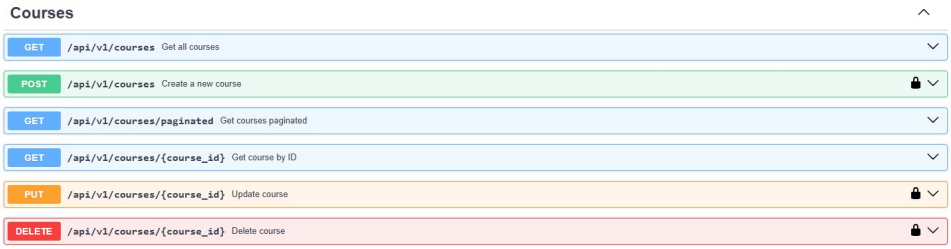


Figure 14: Course CRUD endpoints documentation showing all available operations (POST, GET, PUT, DELETE) with their respective paths and authentication requirements.

Assignment Endpoints The assignment endpoints manage the many-to-many relationships between professors and courses, implementing the core linking functionality described in user story US_CF_03.

- `POST /api/v1/assignments` — Create a professor-course assignment. Requires authentication. Validates that both professor and course exist. Enforces unique constraint preventing duplicate assignments. Returns 201 Created with assignment details.
- `GET /api/v1/assignments` — List all assignments with optional filtering by professor ID or course ID. Supports pagination. Returns assignments with embedded professor and course information.
- `GET /api/v1/assignments/{id}` — Retrieve specific assignment by ID. Returns assignment details including full professor and course objects.
- `PUT /api/v1/assignments/{id}` — Update assignment status. Requires authentication. Allows changing status between active, inactive, pending, and completed. Returns updated assignment.
- `DELETE /api/v1/assignments/{id}` — Remove professor-course assignment. Requires authentication. Returns 204 No Content on success.

Figure 15 displays the assignment creation endpoint documentation. The request schema shows the required fields (professor_id, course_id, status) and their data types. The response schema includes the complete assignment object with embedded professor and course details, demonstrating the API's approach to reducing client-side requests by returning related data in a single response.

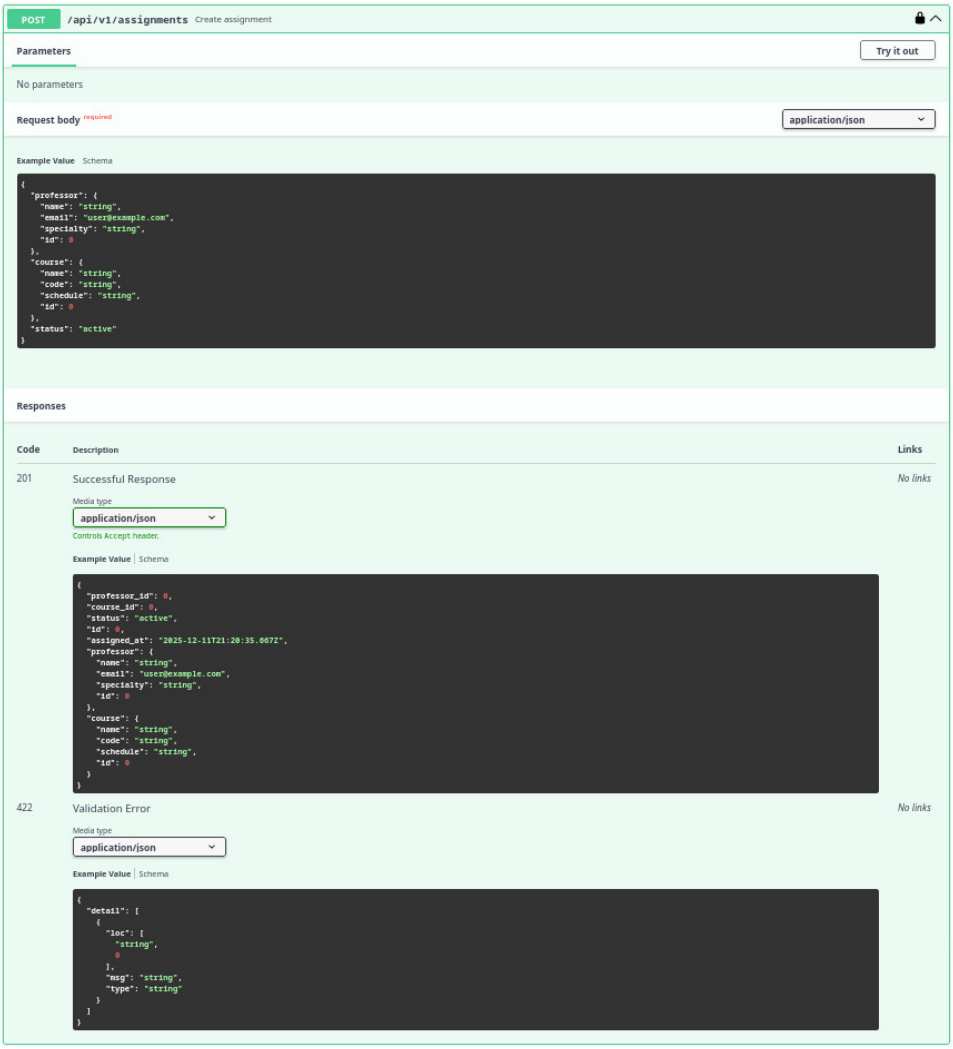


Figure 15: Assignment creation endpoint showing request schema with professor_id, course_id, and status fields, along with the nested response structure including professor and course details.

Search Endpoints While not explicitly listed in the core CRUD operations, the business logic service implements specialized search endpoints that support user story US_CF_04. These endpoints optimize query performance through database indexes and implement the bidirectional search functionality.

- `GET /api/v1/search/courses-by-professor?name=<professor_name>` — Search courses taught by a specific professor. Supports partial name matching and case-insensitive search. Returns list of courses with professor details.

- `GET /api/v1/search/professors-by-course?name=<course_name>` — Search professors teaching a specific course. Supports partial course name or code matching. Returns list of professors assigned to matching courses.

4.5.3 API Design Principles and Error Handling

Both services follow consistent design principles that enhance usability and maintainability. All endpoints return structured JSON responses with predictable formats. Error responses include human-readable messages, error codes, and timestamps to facilitate debugging. Validation errors return 400 Bad Request with detailed field-level error descriptions specifying which fields failed validation and why.

Authentication failures return 401 Unauthorized with clear indication that credentials are invalid or tokens have expired. Authorization failures (attempting to access admin endpoints without proper tokens) return 403 Forbidden. Resource not found scenarios return 404 Not Found with the resource type and ID that could not be located.

Pagination follows cursor-based patterns with consistent query parameters across all list endpoints. Responses include metadata about total items, current page, and page size, enabling frontend components to implement efficient pagination controls. Search endpoints support partial matching and case-insensitive queries to improve user experience and reduce friction during course discovery.

The API versioning strategy uses URL path prefixes (`/api/v1/`) to enable future backward-compatible changes without breaking existing integrations. This approach allows the introduction of new API versions while maintaining support for legacy clients during transition periods.

CORS (Cross-Origin Resource Sharing) configuration allows the React frontend to communicate with both backend services during development and production. The authentication service validates JWT tokens issued by itself, while the business logic service validates tokens by verifying signatures using the shared secret key configured during deployment.

Table 4 summarizes the complete API surface across both services, providing a reference for the total number of endpoints and their authentication requirements.

The API design supports the implementation of all user stories documented in Section 4 while providing the flexibility needed for future feature additions. The consistent patterns across endpoints reduce frontend development complexity and enable code reuse

Table 4: API Endpoints Summary

Service	**Endpoints**	**Auth Required**	**Primary Purpose**
Authentication	1	No	JWT token generation
Professors	5	Yes (except GET list/single)	Professor CRUD
Courses	5	Yes (except GET list/single)	Course CRUD
Assignments	5	Yes (except GET list)	Assignment management
Search	2	No	Bidirectional search
Total	**18**		

through shared API client libraries.

4.6 Deployment View

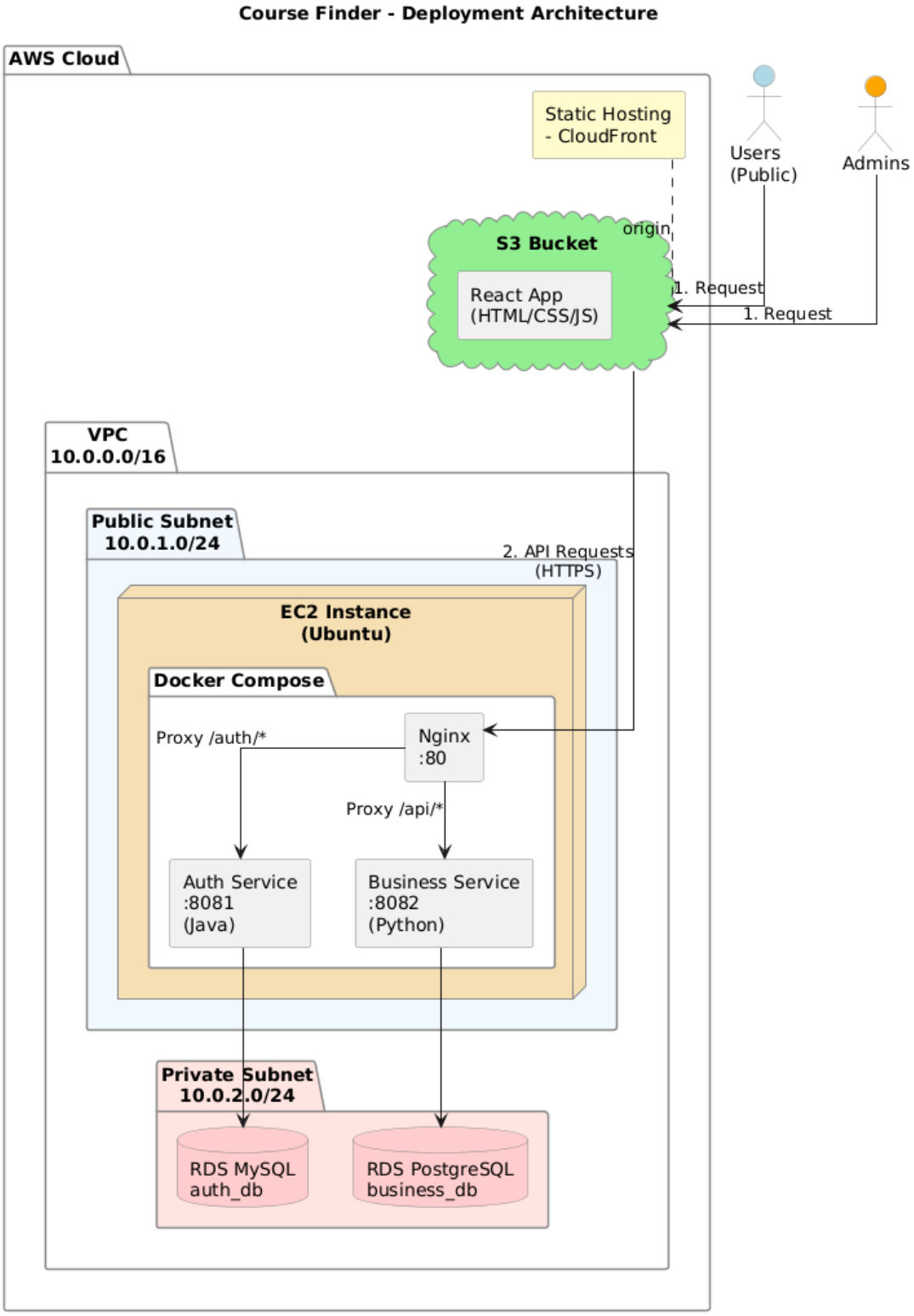


Figure 16: Deployment diagram (v1.0).

5 Implementation and Quality Assurance

5.1 Testing Strategy

The testing strategy for the Course Finder platform follows a multi-layered approach designed to validate functionality at different levels of the system architecture. Tests were implemented after completing the initial development phase, focusing on critical business logic components, API endpoints, user story acceptance criteria, and system performance under load. The overall philosophy prioritizes thorough coverage of core functionality and user-facing features while ensuring that the system meets the quality expectations defined in the acceptance criteria of each user story.

The testing approach focused on meaningful validation of critical paths: unit tests for business logic components related to API endpoints, acceptance tests mapping directly to user stories, and stress tests evaluating system performance and scalability. This pragmatic strategy ensures that tests provide real value in detecting defects and validating requirements.

5.1.1 Unit Testing

Unit tests validate individual components in isolation, ensuring that business logic functions correctly independent of external dependencies such as databases or network services. Mock objects simulate repository and service interactions, allowing precise control over test scenarios and enabling rapid test execution without infrastructure dependencies.

Python Backend - Business Logic Service The business logic service implemented includes comprehensive unit tests using pytest. Tests cover the core service layer components responsible for CRUD operations on professors, courses, and assignments. The test suite uses fixtures to provide consistent test data and mock repository objects that simulate database operations without requiring actual PostgreSQL connections.

Table 5 summarizes the unit test coverage for the Python backend services. Each service class includes dedicated test classes organized by operation type (Create, Read, Update, Delete), following the Arrange-Act-Assert pattern for clarity and maintainability.

The ProfessorService tests validate critical scenarios including successful professor creation with unique email addresses, prevention of duplicate email registration through HTTPException with 409 Conflict status, retrieval of individual professors and paginated lists, handling of non-existent professor queries with 404 Not Found responses, successful updates with field modifications, and cascade deletion behavior. Each test uses mock

Table 5: Python Backend Unit Tests Summary

Service	Tests	Scenarios Covered
ProfessorService	13	Create (success, duplicate email), Read (by ID, all, paginated, not found), Update (success, not found, duplicate email), Delete (success, not found)
CourseService	14	Create (success, duplicate code), Read (by ID, all, paginated, empty list, not found), Update (success, not found, duplicate code), Delete (success, not found)
Total	**27**	

repository objects that return predefined data, allowing precise verification of service layer logic without database dependencies.

Similarly, the CourseService tests cover analogous scenarios for course entities, with particular emphasis on course code uniqueness validation and schedule information handling. Tests verify that the service correctly enforces business rules such as preventing duplicate course codes, returning appropriate error responses for invalid operations, and properly transforming domain models into response schemas.

Mock fixtures defined in `conftest.py` provide reusable test data including sample professor and course objects with realistic attributes, schema instances for create and update operations, and pre-configured mock repository objects with all necessary methods. This fixture-based approach reduces test code duplication and ensures consistency across test cases.

Java Backend - Authentication Service The authentication service implemented includes unit tests using JUnit 5 to validate authentication logic and JWT token generation. Tests focus on two critical components: the authentication service responsible for credential validation and the JWT utility class responsible for token generation and validation.

The AuthService tests cover three primary scenarios using mocked dependencies. First, successful login tests verify that valid username and password combinations result in JWT token generation, with the AdminRepository mock returning a valid admin

entity and the PasswordEncoder mock confirming password match. Second, failed login due to incorrect password tests verify that the service throws appropriate authentication exceptions when the password encoder indicates a mismatch, even when the user exists in the repository. Third, failed login due to non-existent user tests validate that queries for unknown usernames result in authentication failures with appropriate error messages.

The JwtUtil tests verify token generation correctness by creating tokens with test data and validating the resulting JWT structure. Tests confirm that generated tokens are non-null, contain the expected username as the subject claim, and include valid expiration timestamps set according to the configured token lifetime. These tests use temporary RSA key pairs instead of production keys, ensuring security while maintaining test validity.

Both test suites use extensive mocking to isolate the components under test from external dependencies. The AdminRepository is mocked to return predefined admin entities without database queries. The PasswordEncoder is mocked to control password matching behavior deterministically. The JwtUtil mock (when testing AuthService) provides predictable token values. This isolation ensures that tests execute quickly and reliably regardless of external system state.

5.1.2 Acceptance Testing with Cucumber

Acceptance tests validate that the implemented system satisfies the user stories and acceptance criteria defined during requirements analysis. These tests use Cucumber to express scenarios in Gherkin syntax, providing readable specifications that map directly to user story acceptance criteria and can be executed as automated tests.

Four feature files cover the core user stories documented in Section 4. Each feature includes multiple scenarios that validate both successful operations and error handling for edge cases.

Feature: Registration of New Teachers (US_CF_01) This feature validates professor registration functionality through four scenarios. The first scenario verifies that administrators can access the registration form from the main panel. The second scenario validates successful professor creation when all required fields (name, email, specialty) are provided with valid data, confirming that the system displays success messages and updates the professor list without page reload. The third scenario tests validation behavior when required fields are left empty, ensuring appropriate error messages guide administrators to correct the input. The fourth scenario verifies duplicate prevention by attempting to register a professor with an email or ID that already exists, confirming

that the system prevents creation and displays clear error messages.

Feature: Registration of New Subjects (US_CF_02) This feature validates course registration through five scenarios covering similar patterns to professor registration. Scenarios test form access, successful registration with valid data, required field validation, duplicate course prevention based on course code, and optional teacher assignment during course creation. The teacher assignment scenario specifically validates that when an administrator assigns an existing teacher during course creation, the relationship is correctly stored in the database and visible in both professor and course views.

Feature: Assignment of Professors to Courses (US_CF_03) This feature addresses the core relationship management functionality through seven scenarios. Initial scenarios validate interface access and successful assignment creation with valid professor and course selections. Validation scenarios ensure that missing required fields prevent assignment creation with appropriate error messages. Two scenarios specifically test the many-to-many relationship behavior: one verifying that a single professor can be assigned to multiple courses, and another confirming that a single course can have multiple professors assigned. A duplicate prevention scenario validates that the system enforces the unique constraint on professor-course pairs, preventing redundant assignments. Finally, a deletion scenario confirms that administrators can remove assignments and that the system correctly updates the interface.

Feature: Search Courses by Professor and Professors by Course (US_CF_04) This feature validates the primary end-user search functionality through four scenarios. The first scenario tests searching courses by professor name, verifying that all courses taught by the specified professor appear in results. The second scenario validates the inverse operation: searching professors by course name. The third scenario specifically tests partial matching and case-insensitive search behavior, ensuring that users can find results even with incomplete or lowercase search terms. The fourth scenario validates graceful handling of searches with no matches, confirming that the system displays friendly "No results found" messages rather than errors or empty pages.

Table 6 summarizes the acceptance test coverage across all features, showing the direct mapping from user stories to automated test scenarios.

All 20 acceptance test scenarios pass successfully, confirming that the implemented

Table 6: Cucumber Acceptance Test Scenarios Summary

Feature / User Story	Scenarios	Status
US_CF_01: Registration of Teachers	4	Passing
US_CF_02: Registration of Subjects	5	Passing
US_CF_03: Assignment Management	7	Passing
US_CF_04: Bidirectional Search	4	Passing
Total	**20**	**All Passing**

system satisfies the requirements documented in the user stories. The Gherkin scenarios serve dual purposes: they provide executable specifications that validate system behavior automatically, and they function as living documentation that stakeholders can read to understand system capabilities without technical expertise.

The acceptance tests execute against the full application stack including frontend, both backend services, and test databases populated with seed data. This end-to-end validation ensures that component integration functions correctly and that the user experience matches specifications. Test execution occurs both during local development and as part of the CI/CD pipeline, providing continuous validation that new changes do not break existing functionality.

5.1.3 Stress Testing with JMeter

Stress testing evaluates system performance and scalability under realistic and peak load conditions. These tests identify performance bottlenecks, validate response time requirements specified in user stories, and ensure that the system remains stable and responsive when serving multiple concurrent users. Stress tests using Apache JMeter are currently under active development and will be completed in the next project phase.

The planned stress testing strategy will focus on critical endpoints identified through user story analysis. Primary targets include the authentication endpoint (login), search endpoints (courses by professor, professors by course), and administrative CRUD operations (professor creation, course creation, assignment management). Test scenarios will simulate realistic usage patterns with varying levels of concurrency to identify system behavior under normal load, peak load, and stress conditions.

Initial test plans specify scenarios with 50, 100, and 500 concurrent users executing typical workflows over sustained periods (5-10 minutes). Metrics to be collected include average response time, 95th percentile response time, throughput (requests per second), error rate, and resource utilization (CPU, memory, database connections). These met-

rics will validate whether the system meets the performance requirement specified in US_CF_04 and US_CF_05 (response times under two seconds for average query loads).

Results from stress testing will inform infrastructure sizing decisions for production deployment, identify optimization opportunities in database queries or API implementations, and validate that the system architecture can scale to support the anticipated user base. Performance baselines established through these tests will guide future capacity planning and serve as regression benchmarks for subsequent releases.

5.1.4 Testing Infrastructure and Automation

The testing infrastructure integrates with the CI/CD pipeline implemented through GitHub Actions. Unit tests execute automatically on every commit and pull request, providing immediate feedback to developers about test failures or regressions.

Acceptance tests execute as part of the integration testing phase after successful unit test completion. These tests require more setup time as they spin up Docker containers for both backend services and test databases, populate seed data, and initialize the frontend application. The complete acceptance test suite executes in approximately 5-10 minutes, providing thorough validation before deployment to staging environments.

Test results are reported through the GitHub Actions interface, with detailed logs available for investigation when failures occur. Failed tests block pull request merges, ensuring that defects are identified and resolved before integration into the main codebase. This continuous validation approach maintains code quality and prevents regression of previously working functionality.

Local development environments support test execution through simple commands (`pytest` for Python unit tests, `mvn test` for Java unit tests, `npm run test:cucumber` for acceptance tests), enabling developers to validate changes before committing. Mock-based unit tests execute extremely quickly in local environments, supporting test-driven development workflows even for components with complex dependencies.

5.2 CI/CD Pipeline

The Course Finder platform implements a comprehensive Continuous Integration and Continuous Deployment (CI/CD) pipeline using GitHub Actions for automated testing and building, with Jenkins handling deployment orchestration. This automated pipeline ensures code quality, catches defects early, and accelerates the development cycle by providing rapid feedback on every code change. The pipeline architecture follows modern

DevOps practices including containerization with Docker, parallel job execution, and infrastructure as code principles.

5.2.1 Pipeline Architecture and Workflow Organization

The CI/CD infrastructure consists of four GitHub Actions workflows organized by component and responsibility. Table 7 summarizes the workflow structure and execution triggers.

Table 7: GitHub Actions Workflows Configuration

Workflow	**Triggers**	**Components**
Workshop 4 - CI Pipeline	Push/PR to main, master, develop; Manual dispatch	All three services (parallel execution)
Workshop 4 - Frontend CI	Push/PR affecting frontend paths; Manual dispatch	Lint → Build (sequential)
Workshop 4 - Java Backend CI	Push/PR affecting Java backend paths; Manual dispatch	Test, Lint (parallel) → Build
Workshop 4 - Python Backend CI	Push/PR affecting Python backend paths; Manual dispatch	Test, Lint (parallel) → Build

The primary workflow (`Workshop 4 - CI Pipeline`) executes all service pipelines in parallel when changes affect the `Workshop-4/` directory, providing comprehensive validation across the entire application stack. Individual service workflows target specific paths, triggering only when relevant code changes, optimizing CI resource usage and execution time. All workflows support manual triggering through `workflow_dispatch`, enabling on-demand validation and testing.

Path-based filtering ensures efficient resource utilization by preventing unnecessary builds. For example, changes exclusively to frontend code trigger only the frontend workflow, while changes to shared configuration or multiple components trigger the comprehensive pipeline. This selective execution reduces average pipeline time and GitHub Actions compute minutes consumption.

5.2.2 Frontend Pipeline

The frontend pipeline builds and validates the React-based user interface through two sequential stages: linting and building. The pipeline uses Node.js 18 with npm caching

to accelerate dependency installation across workflow runs.

The lint stage executes ESLint with the project's configured rules, validating code style consistency, identifying potential bugs through static analysis, and enforcing React best practices. ESLint checks cover JavaScript syntax errors, unused variables, improper hook usage, and accessibility violations. Failed linting blocks the build stage, ensuring that only style-compliant code progresses through the pipeline.

The build stage compiles the React application using Vite, producing an optimized production bundle with minified JavaScript, CSS extraction, and asset optimization. The build process validates that all imports resolve correctly, TypeScript types compile without errors, and environment variable references are satisfied. Successful builds generate distributable artifacts in the `dist/` directory ready for containerization.

Figure 17 shows the performance metrics for the frontend workflows. The average execution time of 35 seconds makes this the fastest component in the pipeline, with the majority of time spent on dependency installation and Vite compilation. The 67% failure rate reflects active development with frequent linting issues that are caught and corrected before merge.

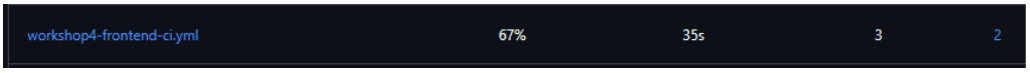


Figure 17: Frontend CI workflow performance metrics showing average run time of 35 seconds and 67% job failure rate primarily due to linting violations caught during development.

5.2.3 Java Backend Pipeline - Authentication Service

The Java backend pipeline validates the Spring Boot authentication service through three stages: testing, static analysis, and building. The pipeline uses JDK 21 with Maven caching for dependency management.

The test stage executes JUnit 5 unit tests in an environment with a containerized MySQL 8.0 database service. GitHub Actions services provide the MySQL instance with health checks ensuring database readiness before test execution. Tests run with environment variables configuring test database connection parameters and JWT secret keys. The Maven command `mvn test` executes the complete test suite covering authentication logic, password encoding, JWT generation, and repository operations.

The static analysis stage runs Maven Checkstyle with the project's configured ruleset. Checkstyle validates Java code formatting, naming conventions, import organization, and complexity metrics. The stage uses `continue-on-error: true`, treating Checkstyle violations as warnings rather than blocking failures. This configuration acknowledges that style violations are less critical than functional defects while still providing visibility into code quality trends.

The build stage compiles the application and packages it as an executable JAR file using `mvn clean package -DskipTests`. The `-DskipTests` flag prevents redundant test execution since tests already ran in the dedicated test stage. The resulting JAR includes all dependencies and Spring Boot's embedded Tomcat server, producing a self-contained artifact ready for containerization.

Figure 18 illustrates the Java backend pipeline performance. The average execution time of 1 minute 11 seconds primarily reflects Maven dependency resolution and test execution against the MySQL container. The 67% failure rate indicates active development with test failures caught during feature implementation before code review.

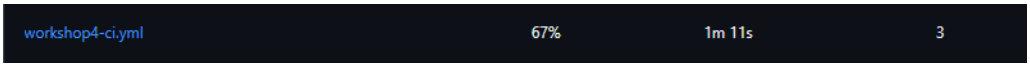


Figure 18: Java backend CI workflow performance metrics showing average run time of 1 minute 11 seconds and 67% job failure rate reflecting test failures during development.

5.2.4 Python Backend Pipeline - Business Logic Service

The Python backend pipeline validates the FastAPI business logic service through testing, linting, and building stages. The pipeline uses Python 3.11 with Poetry for dependency management and a containerized PostgreSQL 15 database for testing.

The test stage installs Poetry, creates the project's virtual environment, generates temporary RSA key pairs for JWT operations, and executes pytest against a PostgreSQL container. The database service includes health checks ensuring PostgreSQL readiness before test execution. Tests run with environment variables configuring database connection parameters. The `poetry run pytest tests/` command discovers and executes all test modules covering service layer operations, repository interactions, and schema validations.

The lint stage runs Pylint with the project's configuration, analyzing code quality, detecting potential errors, and enforcing PEP 8 style guidelines. Similar to the Java pipeline,

Pylint uses `continue-on-error: true`, treating style violations as non-blocking warnings. This pragmatic approach prioritizes functional correctness while maintaining visibility into code quality metrics.

The build stage uses `poetry build` to create distributable Python wheel and source distribution packages. While these packages are not directly deployed (the Docker image includes source code instead), the build validates that the project structure is correct and dependencies are properly declared in `pyproject.toml`.

Performance metrics for the Python backend show reasonable execution times balanced against thorough validation. The testing stage including PostgreSQL initialization, Poetry environment setup, and test execution completes within acceptable timeframes for developer feedback.

5.2.5 Containerization with Docker

Each service includes a multi-stage Dockerfile optimizing for image size, build speed, and security. The multi-stage approach separates build dependencies from runtime requirements, significantly reducing final image sizes and attack surfaces.

Python Backend Dockerfile The Python backend uses a two-stage build process based on `python:3.11-slim`. The builder stage installs Poetry, system build dependencies (gcc, libpq-dev), and project dependencies in a virtual environment. Poetry's configuration creates the virtual environment within the project directory (`POETRY_VIRTUALENVS_IN_PROJECT=1`), simplifying the copy to the runtime stage.

The runtime stage uses a fresh `python:3.11-slim` base, copies only the virtual environment from the builder (avoiding Poetry and build tools), installs runtime system packages (libpq5 for PostgreSQL client), and creates a non-root user (`appuser`) for security. The `HEALTHCHECK` directive defines an endpoint for container orchestration to monitor service health. The final image exposes port 8000 and starts the application with Uvicorn ASGI server.

Java Backend Dockerfile The Java backend uses `maven:3.9.11-amazoncorretto-25-alpine` for building and `amazoncorretto:25-alpine` for runtime. The builder stage copies the POM file, downloads dependencies (cached by Maven), copies source code, and packages the application with `mvn clean package -DskipTests`. This ordering maximizes Docker layer caching—dependency layers rebuild only when POM changes.

The runtime stage copies only the JAR file from the builder, avoiding Maven, source code, and build artifacts in the final image. Amazon Corretto provides an optimized OpenJDK distribution with performance improvements and long-term support. The image exposes port 8080 and uses `ENTRYPOINT` to start the Spring Boot application.

Frontend Dockerfile The frontend uses `node:18-alpine` for building and `nginx:alpine` for serving. The builder stage installs dependencies, copies source code, and executes `npm run build` to generate the production Vite bundle. The runtime stage uses Nginx Alpine (a minimal 20MB base image) to serve static files efficiently.

The Dockerfile copies the Vite output (`dist/`) to Nginx's HTML root and applies a custom `nginx.conf` configuring routing, caching headers, and compression. This approach produces extremely small images (under 50MB) optimized for serving static content with high performance and low resource consumption.

5.2.6 Container Orchestration with Docker Compose

The `docker-compose.yml` file orchestrates all services and their dependencies for local development and testing environments. The composition includes five services: two databases (MySQL, PostgreSQL), two backend services, and the frontend, all connected through a custom bridge network.

Database services use official images (mysql:8.0, postgres:15-alpine) with environment variables configuring credentials, persistent volumes for data durability, and health checks ensuring readiness before dependent services start. The MySQL service mounts an initialization script (`schema_db.sql`) automatically creating the authentication database schema. Similarly, the PostgreSQL service initializes the business logic database schema.

Backend services depend on their respective databases with `condition: service_healthy`, ensuring proper startup ordering. The Java backend connects to MySQL using JDBC, while the Python backend connects to PostgreSQL using asyncpg. Both services receive database connection parameters and secrets through environment variables, supporting the twelve-factor app configuration methodology.

The frontend service depends on both backends and exposes port 80, providing the entry point for browser access. The custom bridge network (`workshop-network`) enables service discovery through DNS—services reference each other by container name (e.g., `java-backend-db:3306`) rather than IP addresses.

Named volumes (`mysql_data`, `postgres_data`) persist database state across container restarts, preventing data loss during development. Environment variables use defaults with override capability through `.env` files, supporting different configurations for development, testing, and production environments.

5.2.7 Pipeline Performance and Reliability Metrics

GitHub Actions Insights provides comprehensive metrics tracking pipeline performance and reliability. Figure 19 shows the aggregate usage across all workflows for the current month.

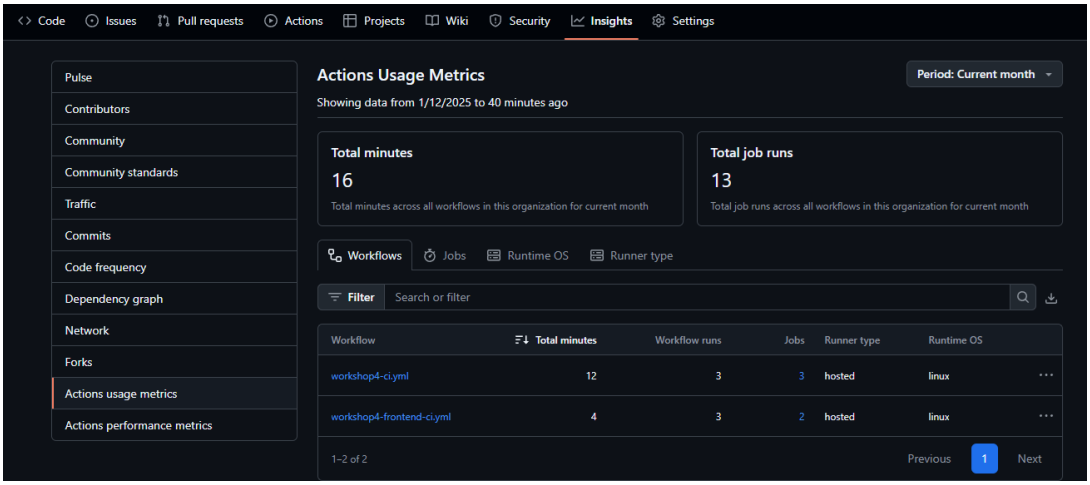


Figure 19: GitHub Actions usage metrics showing 16 total minutes consumed and 13 job runs across all workflows in the current month.

The metrics reveal efficient resource utilization with 16 total minutes across 13 job runs, averaging approximately 1.2 minutes per job. This efficiency results from path-based filtering, parallel job execution, and aggressive caching strategies. The primary workflow (`workshop4-ci.yml`) accounts for 12 minutes across 3 runs (4 minutes average), while individual service workflows consume less time due to focused scope.

Figure 20 presents detailed performance metrics including average job run time (42 seconds), average queue time (2 seconds), job failure rate (31%), and failed job usage (4 minutes).

The 31% overall failure rate reflects normal development activity where code changes trigger pipeline runs that may fail due to test failures, linting violations, or build errors. Failed jobs are caught before code review and merge, fulfilling the primary purpose of continuous integration: detecting defects early when they are cheapest to fix. The majority of failures occur during active feature development, with the failure rate dropping

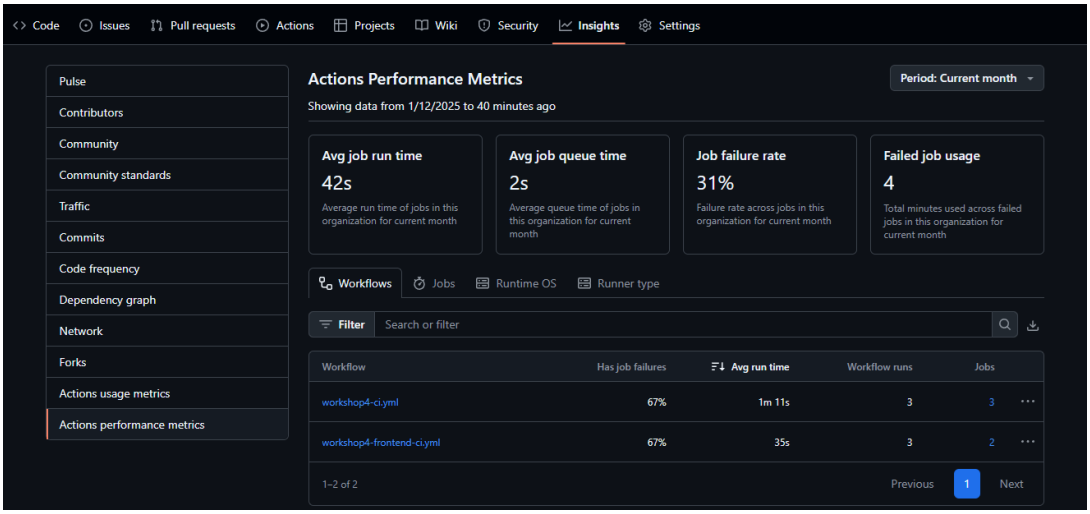


Figure 20: Pipeline performance metrics showing 42-second average runtime, 2-second queue time, 31% failure rate, and 4 minutes consumed by failed jobs.

significantly as code approaches merge readiness.

Figure 21 shows the recent workflow execution history with status indicators. The mix of successful (green checkmarks) and failed (red X marks) runs demonstrates the pipeline's effectiveness at catching issues during development while allowing clean code to proceed.

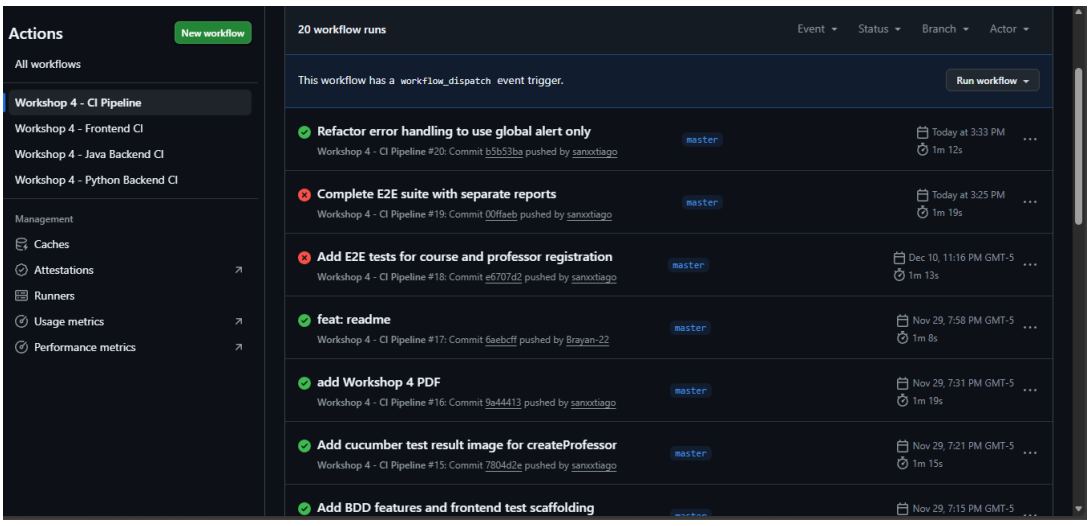


Figure 21: Recent workflow runs showing successful and failed executions with duration times ranging from 1 minute 8 seconds to 1 minute 19 seconds.

Figure 22 illustrates the pull request check status for a specific commit. Multiple workflow checks run in parallel, with clear status indicators showing which checks passed (Java Backend, Python Backend) and which failed (Frontend, Frontend Lint). This granular feedback enables developers to identify and address specific issues quickly.

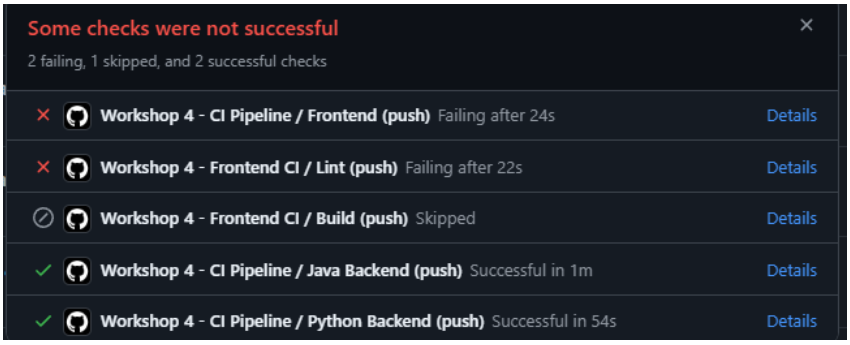


Figure 22: Pull request checks showing parallel workflow execution with 2 failing checks (Frontend), 1 skipped check (Frontend Build), and 2 successful checks (Java Backend, Python Backend).

Pull requests cannot be merged until all required checks pass, enforcing code quality gates and preventing regression. This protection mechanism ensures that the main branch maintains a stable, deployable state at all times.

5.2.8 Continuous Deployment with Jenkins

The continuous deployment pipeline extends beyond GitHub Actions to include Jenkins-based deployment orchestration. Jenkins monitors the repository for successful builds on the main branch and triggers automated deployment workflows. The deployment infrastructure leverages the Docker images produced by the CI pipeline, deploying containerized services to target environments.

Jenkins deployment pipelines handle environment-specific configuration, secret injection, database migrations, health checks, and rollback procedures. The deployment automation reduces manual errors, accelerates release cycles, and provides audit trails for compliance and troubleshooting. Detailed deployment workflows and infrastructure specifications will be documented in subsequent project phases as the deployment architecture matures.

5.2.9 Benefits and Impact of CI/CD Implementation

The implemented CI/CD pipeline provides significant benefits to the development process. Automated testing catches defects within minutes of code changes, dramatically reducing the feedback cycle compared to manual testing. Parallel job execution across multiple services optimizes total pipeline time, providing rapid feedback without sacrificing thoroughness. Path-based filtering and dependency caching minimize unnecessary

work, conserving CI resources and reducing costs.

The pipeline enforces quality gates through required status checks on pull requests, preventing low-quality code from reaching the main branch. Developers receive immediate, actionable feedback through GitHub's UI, enabling quick iteration and correction. The combination of unit tests, linting, and build validation ensures multiple layers of quality control without requiring manual intervention.

Containerization through Docker provides consistency across development, testing, and production environments, eliminating "works on my machine" issues. Docker Compose simplifies local development setup, enabling new team members to start contributing quickly. The multi-stage Dockerfile approach balances build convenience with production image efficiency and security.

The CI/CD infrastructure serves as the foundation for continuous delivery practices, enabling the team to deploy changes rapidly and reliably. As the project matures, additional stages including integration testing, security scanning, and automated deployment to staging environments can be integrated into the existing pipeline structure without fundamental architectural changes.

5.3 Code Quality and Standards

Maintaining consistent code quality across multiple programming languages and frameworks requires well-defined standards, automated enforcement, and clear documentation. The Course Finder platform implements quality standards through configuration-driven linting tools, structured project organization, comprehensive documentation, and automated quality gates integrated into the CI/CD pipeline. These practices ensure code maintainability, facilitate team collaboration, and reduce technical debt accumulation.

5.3.1 Frontend Code Standards

The frontend application follows modern JavaScript and React best practices enforced through ESLint with TypeScript support. The linting configuration extends multiple recommended rule sets providing comprehensive coverage of potential issues.

The ESLint configuration (`eslint.config.js`) uses the new flat config format introduced in ESLint 9, defining rules through composable configuration objects. The configuration extends four base rule sets: `js.configs.recommended` for core JavaScript best practices, `tseslint.configs.recommended` for TypeScript-specific rules, `reactHooks.configs['recommended-latest']` for React Hooks usage patterns, and `reactRefresh.`

`configs.vite` for Vite-specific optimizations including Fast Refresh compatibility.

The language options specify ECMAScript 2020 features and browser globals, ensuring that the linter recognizes modern JavaScript syntax and browser-specific APIs without false positives. The configuration applies only to TypeScript files (`**/*.ts,tsx`), ignoring the `dist/` directory containing build artifacts.

Key enforced standards include proper React Hooks dependency arrays preventing stale closure bugs, exhaustive dependency declarations ensuring effects run when expected, component naming conventions for clarity and tooling support, proper TypeScript typing avoiding `any` types where possible, and unused variable detection preventing dead code accumulation. The ESLint check runs as a required stage in the CI pipeline, blocking merges when violations are detected.

The frontend follows a feature-based project structure organizing code by domain concern rather than technical layer. Components, hooks, services, and types related to a specific feature coexist in the same directory, improving cohesion and reducing coupling. Shared utilities and components reside in dedicated directories accessible across features. This organization pattern scales effectively as the application grows and facilitates code discovery for new team members.

5.3.2 Authentication Backend Code Standards

The Java authentication service follows conventional Spring Boot project structure with clear separation between technical layers. Figure 23 illustrates the package organization.

The layered package structure includes `config` for Spring configuration classes and beans, `controller` for REST API endpoints annotated with `@RestController`, `dto` (Data Transfer Objects) for request/response payloads, `entity` for JPA entities representing database tables, `repository` for Spring Data JPA repositories, `security` for authentication and authorization components including JWT utilities, and `service` for business logic implementation.

This organization follows Spring Boot conventions making the project structure immediately familiar to Java developers. Each package has a single, well-defined responsibility following the Single Responsibility Principle. Dependencies flow from controllers through services to repositories, maintaining proper layering without circular dependencies.

Naming conventions follow Java standards: classes use PascalCase, methods and vari-

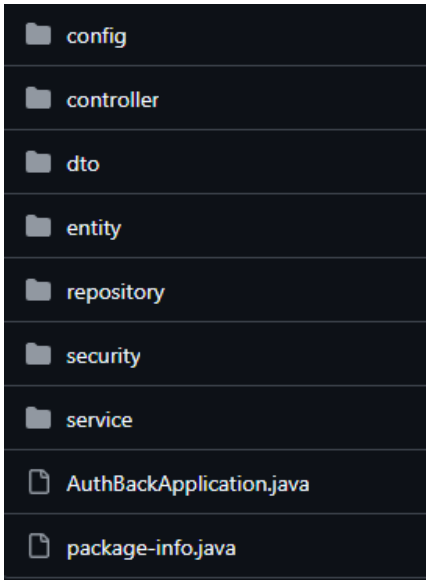


Figure 23: Java backend package structure organized by technical layers: config, controller, dto, entity, repository, security, and service.

ables use camelCase, constants use UPPER_SNAKE_CASE, interfaces describe capabilities (e.g., `Repository`, `Service`). Package names use lowercase without underscores.

5.3.3 Business logic Backend Code Standards

The Python business logic service follows FastAPI best practices with a modular architecture emphasizing clear separation of concerns. The project uses Poetry for dependency management and modern Python tooling for code quality enforcement.

The `pyproject.toml` file defines project metadata, dependencies, development tools, and quality tool configurations. Table 8 summarizes the development tools and their purposes.

The project structure organizes code by technical concern with clear boundaries between layers. The `api/` package contains route definitions and dependencies, `core/` includes configuration and database setup, `models/` defines SQLAlchemy ORM models, `repositories/` implements data access patterns, `schemas/` contains Pydantic models for validation and serialization, `services/` encapsulates business logic, and `specifications/` implements the Specification pattern for complex queries. This architecture maintains clear dependency flow: routes depend on services, services depend on repositories, and repositories depend on models.

Ruff serves as the primary code quality tool, combining linting and formatting in

Table 8: Python Backend Development Tools

Tool	Version	Purpose
pytest	~7.4.3	Unit testing framework with fixtures and parametrization
pytest-asyncio	~0.21.1	Async test support for FastAPI endpoints
pytest-cov	~7.0.0	Code coverage measurement and reporting
ruff	~0.1.0	Fast Python linter and formatter (Rust-based)
mypy	~1.7.0	Static type checker for gradual typing

a single fast tool written in Rust. The configuration targets Python 3.11 with a 100-character line length. Selected rule sets include `E` (pycodestyle errors), `F` (Pyflakes), `I` (isort import sorting), `N` (PEP 8 naming conventions), and `W` (pycodestyle warnings). The `E501` rule (line too long) is explicitly ignored, relying on Ruff's formatter to handle line length pragmatically.

Mypy provides optional static type checking with `disallow_untyped_defs: false`, allowing gradual adoption of type hints without blocking development. The `warn_return_any` and `warn_unused_configs` options catch common type-related issues without requiring complete type coverage. This pragmatic approach balances type safety benefits against the overhead of comprehensive type annotations.

Naming conventions follow PEP 8: modules use snake_case, classes use PascalCase, functions and variables use snake_case, constants use UPPER_SNAKE_CASE, and "internal" names use leading underscore. Async functions include `async` in their name when the async nature is not obvious from context, improving code readability.

5.3.4 Documentation Standards and API Specification

Each service includes comprehensive README documentation following consistent structure across projects. The Python backend README (representative of the documentation standard) includes quick start instructions with dependency installation, environment configuration, database initialization, and application startup commands. The documentation specifies exact versions of required tools and provides platform-specific installation instructions.

API documentation leverages OpenAPI (formerly Swagger) specifications automatically generated by FastAPI and Spring Boot frameworks. The Python backend exposes interactive documentation at `/api/v1/docs` (Swagger UI) and `/api/v1/redoc` (ReDoc), providing developers with self-service API exploration without requiring separate documentation maintenance. The Java backend similarly exposes Swagger UI at `/swagger-ui/index.html`.

The OpenAPI specifications include endpoint descriptions, request/response schemas with validation rules, authentication requirements, example payloads, and error response formats. This machine-readable documentation enables automatic client generation, API testing tools, and developer onboarding without manual documentation updates. The specifications stay synchronized with code automatically as framework annotations and type hints directly generate the OpenAPI schemas.

Project structure documentation in README files includes ASCII tree diagrams showing directory organization and brief descriptions of each package's purpose. This high-level view helps developers navigate unfamiliar codebases and understand architectural boundaries. Each README also documents core endpoints by category (Professors, Courses, Assignments), providing quick reference for API capabilities.

Security documentation explains authentication mechanisms, JWT token requirements, public key configuration, and protected endpoint usage. Code examples demonstrate how to use authentication dependencies in new endpoints, reducing friction when extending the API. Environment variable documentation specifies required configuration with examples and security recommendations.

5.3.5 Automated Quality Gates and Enforcement

The CI/CD pipeline enforces quality standards through automated checks that must pass before code can be merged. Pull requests require three passing checks: all unit tests must execute successfully with no failures, linting must complete without blocking violations, and builds must produce valid artifacts. GitHub branch protection rules prevent merging pull requests with failing checks, ensuring that the main branch maintains consistent quality standards.

The test requirement validates functional correctness through the comprehensive unit test suites documented in Section 5. Test failures indicate regression or incomplete implementation, requiring correction before merge. The fast test execution (under 2 minutes total across all services) provides rapid feedback without impeding development velocity.

The linting requirement catches style violations, potential bugs, and code quality issues before code review. Frontend linting failures block merges, enforcing consistent React and TypeScript patterns. Backend linting (Checkstyle for Java, Ruff for Python) runs with `continue-on-error: true`, generating reports without blocking merges. This pragmatic configuration acknowledges that style consistency matters but should not prevent delivery of correct functionality. Teams review linting reports during sprint retrospectives, gradually improving code quality through focused refactoring efforts.

The build requirement validates that code compiles, dependencies resolve, and packaging succeeds. Build failures indicate missing dependencies, syntax errors, or configuration issues that would prevent deployment. Successful builds produce Docker images or application artifacts ready for deployment.

These automated quality gates reduce manual review burden by catching objective issues (syntax errors, test failures, obvious style violations) before human reviewers examine code. Reviewers focus on architectural decisions, business logic correctness, and maintainability rather than mechanical issues the CI pipeline detects automatically.

5.3.6 Testing Standards and Coverage

The testing strategy documented in Section 5 establishes standards for test organization, naming, and coverage. Each service includes unit tests for business logic components with comprehensive scenario coverage including success paths, error handling, validation failures, and edge cases.

Test organization follows consistent patterns across languages. Python tests use pytest with class-based grouping organizing related scenarios. Java tests use JUnit 5 with similar class organization and descriptive method names. Test fixture management through pytest fixtures and test data builders ensures consistent, maintainable test data across scenarios.

Test naming conventions emphasize clarity over brevity. Test method names describe the scenario and expected outcome: `test_create_professor_success`, `test_create_professor_duplicate_email`. This naming pattern makes test failures immediately understandable without examining test code, accelerating debugging.

Mock objects simulate external dependencies (databases, repositories) enabling fast, isolated unit tests. The consistent use of mocks across tests maintains test indepen-

dence—tests can run in any order without side effects. This isolation supports parallel test execution and reliable CI pipeline results.

5.3.7 Impact on Development Workflow

The implemented quality standards and automation significantly impact development efficiency and code quality outcomes. Automated linting catches common mistakes during development before commit, reducing the cost of fixes. IDE integration with ESLint, Checkstyle, and Ruff provides real-time feedback, enabling developers to address issues immediately rather than discovering them in CI failures.

Comprehensive README documentation reduces setup friction. Clear instructions with copy-paste commands enable productive contribution within hours rather than days. API documentation through OpenAPI specifications reduces communication overhead—developers reference interactive documentation rather than asking colleagues about endpoint behaviors.

The layered architecture and clear package structure enforced through conventions reduce cognitive load when navigating unfamiliar code. Developers can predict where functionality resides based on architectural patterns, accelerating debugging and feature development. The consistent structure across services (frontend, authentication backend, business logic backend) enables knowledge transfer—patterns learned in one service apply to others.

Quality gates in the CI/CD pipeline maintain baseline quality without manual enforcement. Developers receive automated feedback rather than depending on manual code review to catch mechanical issues. This automation frees reviewers to focus on higher-level concerns: architectural decisions, business logic correctness, performance implications and security considerations.

The balance between strict enforcement (tests must pass, frontend must lint clean) and pragmatic warnings (backend linting non-blocking) acknowledges that different violation types have different impacts. Functional defects prevent deployment; style inconsistencies reduce maintainability but should not block progress. This focus approach maintains velocity while gradually improving quality metrics.

6 Results and Discussion

The outcomes of Workshops 1 and 2 have established a solid analytical foundation for the project. The main deliverables include the problem definition, user stories with acceptance criteria, CRC cards, architectural and deployment diagrams, and early Web UI mockups. These artifacts provide a coherent view of the system's objectives and structure, enabling the team to validate the feasibility of the proposed solution before implementation.

6.1 Testing Results

6.2 System Performance Evaluation

6.3 Achievement of Objectives

6.4 Limitations and Lessons Learned

7 Conclusions

This first report consolidates the analytical and design foundations of the project, demonstrating coherent progress from requirement elicitation to system architecture definition. The established artifacts and planning ensure a clear path toward implementation, testing, and integration in the following workshop stages.

Glossary

API Application Programming Interface.

CI/CD Continuous Integration / Continuous Deployment.

JWT JSON Web Token.

CRC Class - Responsability - Collaboration Cards